DOCUMENTO TÉCNICO CONSOLIDADO · PARTE 1

La Gran Liga de Apuestas de Medianos EIA

Arquitectura de la plataforma de apuestas mutuas (Pari-Mutuel)

Salchipapa Maker Racing S.A.S.

Versión v1.0 · 27 de octubre de 2026
Consolidado de los artefactos P1-ART-002 a P1-ART-010 y los ADR-001 a ADR-003
Documento de análisis y diseño — no contiene implementación de la plataforma

- Análisis de negocio, requisitos e invariantes
- Arquitectura y componentes (Decisión 1 y descartes)
- Flujo crítico y comunicaciones síncronas
- Mensajería y eventos con Apache Kafka (Decisión 2)
- Datos, persistencia e idempotencia (Decisión 3)
- Escalabilidad, dimensionamiento y manejo de fallos
- Seguridad, RBAC y gobernanza de costos
- ADR-001, ADR-002 y ADR-003
- Anexo A: fuentes y renders de los cuatro diagramas

Índice de contenidos

- ETAPA P1-1: ANÁLISIS DE NEGOCIO Y REQUISITOS
- ETAPA P1-2: ARQUITECTURA Y COMPONENTES
- ETAPA P1-3: FLUJO CRÍTICO Y COMUNICACIÓN SÍNCRONA
- ETAPA P1-4: MENSAJERÍA Y COMUNICACIÓN ASÍNCRONA
- ETAPA P1-5: DATOS, PERSISTENCIA E IDEMPOTENCIA
- ETAPA P1-6: ESCALABILIDAD, DIMENSIONAMIENTO Y MANEJO DE FALLOS
- ETAPA P1-7: SEGURIDAD, CONTROL DE ACCESO Y GOBERNANZA DE COSTOS

Registros de Decisión Arquitectónica (ADR)

- ADR-001: Selección del Estilo Arquitectónico
- ADR-002: Selección de la Tecnología de Mensajería y Event Streaming
- ADR-003: Consistencia Transaccional, Idempotencia y Modelo de Liquidación

Anexo A · Fuentes y renders de los diagramas

- DIAGRAMA GENERAL DE ARQUITECTURA
- DIAGRAMA DE SECUENCIA: FLUJO COMPLETO DE UNA APUESTA
- TOPOLOGÍA DE MENSAJERÍA: APACHE KAFKA
- DIAGRAMA DE CICLO DE VIDA DE UNA CARRERA

ETAPA P1-1: ANÁLISIS DE NEGOCIO Y REQUISITOS

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-002`
Versión: `v1.1`
Estado: `[x]` Verificado y consolidado (P1-1.1 aprobado por usuario; P1-1.2 a P1-1.5 completados por revisión autónoma)

P1-1.1: ACTORES, ROLES Y CAPACIDADES DEL SISTEMA

1. Actores y Roles del Sistema

Distinción entre Requisitos Explícitos del PDF y Propuesta de Asignación de Responsabilidades:

Rol	Elementos Explícitos en el PDF (Fuente: PDF págs. 3, 4, 7, 8, 12)	Propuesta de Asignación / Alcance Técnico (Propuesto a aprobación)
Apostador	• Nombre de rol explícito (pág. 4). • Puede registrarse, iniciar sesión y consultar perfiles (pág. 4). • Puede consultar carreras disponibles y realizar apuestas usando EIA Coins (pág. 2, 3). • Puede realizar varias apuestas en la misma carrera, incluso sobre medianos distintos (pág. 3). • Dispone de billetera virtual para consultar saldo y movimientos (pág. 6). • Recibe notificaciones de eventos (pág. 6-7). • Una apuesta enviada después del cierre es rechazada según reloj del servidor (pág. 3).	• Sujeto a rate limiting perimetral y mecanismos de mitigación de bots automatizados. • Consulta de cuotas dinámicas y ficha de competidores con tolerancia a lecturas eventuales. • Clientes móviles y web operan contra interfaces de consumo público.
Operador de Carrera	• Nombre de rol explícito (pág. 4). • No tiene detalle de funciones taxativas en la pág. 4. Se deduce de la capacidad "Carreras" (pág. 5) y "Resultados y premios" (pág. 6) que registra/interviene en programación, inicio, fin y resultados oficiales.	• **Propuesta:** Interfaz interna autenticada para parametrizar eventos (pistas, distancias, medianos participantes y horarios). • **Propuesta:** Ingreso o confirmación de inicio, fin y captura de foto de llegada/resultado oficial. • **Regla estricta:** El cierre de apuestas NO depende de que el operador presione un botón; se rige estrictamente por la hora programada y el reloj del servidor. • No posee facultades para alterar saldos de billeteras arbitrariamente.
Administrador	• Nombre de rol explícito (pág. 4). • Administrar roles, bloquear usuarios y validar permisos (pág. 4). • Todas las acciones administrativas deben registrarse (pág. 12).	• **Propuesta:** Gestión de usuarios (bloqueo/desbloqueo de cuentas sospechosas identificadas por auditoría). • **Propuesta:** Asignación y revocación de privilegios RBAC. • Principio de mínimo privilegio: sin acceso directo de mutación a balances contables históricos ni mutación de resultados oficiales sellados.
Auditor	• Nombre de rol explícito (pág. 4). • **Permisos exclusivos de solo lectura** (explícito pág. 7). • Consulta registro inalterable de apuestas, movimientos de billetera, premios y cambios de resultados (pág. 7). • Detección de comportamientos sospechosos (múltiples cuentas por dispositivo, apuestas masivas al cierre) (pág. 8). • Reconstrucción histórica de cálculo de premios de cualquier carrera (pág. 8).	• Consola de supervisión analítica y forense conectada a réplicas o almacenes inmutables de auditoría. • Incapacidad total de mutación, inserción o borrado en el plano transaccional.

---

2. Las Nueve Capacidades Principales del Sistema (Fuente: PDF págs. 3-8)

El PDF establece de forma numerada y taxativa exactamente **nueve capacidades principales** (secciones 1 a 9, págs. 4 a 8):

1. **Usuarios y autenticación (PDF pág. 4):** Registrar usuarios, iniciar sesión, consultar perfiles, administrar roles, bloquear usuarios y validar permisos. Protección estricta de contraseñas y datos personales.
2. **Medianos (PDF pág. 4-5):** Almacenar información de cada competidor: nombre, edad, peso, historial de carreras, victorias, derrotas, velocidad promedio y estado actual.
3. **Carreras (PDF pág. 5):** Administrar creación de carreras, programación, participantes, horarios, pistas, inicio de carrera, finalización y resultados oficiales. Comunicación inmediata de cambios de estado a la plataforma.
4. **Apuestas (PDF pág. 5):** Permitir realizar apuestas sobre carreras disponibles. Validaciones previas obligatorias: existencia de usuario, carrera abierta (antes del cierre según servidor), mediano participante, saldo suficiente y ausencia de duplicados. Soporte de miles de apuestas por segundo.
5. **Billetera de EIA Coins (PDF pág. 6):** Billetera virtual por usuario para consultar saldo, descontar EIA Coins al apostar, acreditar premios, consultar movimientos y realizar devoluciones ante carreras canceladas. Invariante: nunca permitir doble gasto de monedas ni saldos negativos.
6. **Resultados y premios (PDF pág. 6):** Recepción del resultado oficial al terminar la carrera: identificar ganador, ubicar apuestas ganadoras, calcular premios (bolsa mutua con 10% retenido por la casa), acreditar EIA Coins, registrar movimientos y generar auditoría. Soporte de más de un millón de apuestas sin procesarlas en una sola petición HTTP.
7. **Notificaciones (PDF pág. 6-7):** Envío de mensajes a usuarios por correo, app móvil y push ante hitos: apuesta aceptada, inicio de carrera, término de carrera, apuesta ganada, acreditación de monedas y carrera cancelada. Aislamiento: su caída no debe impedir registrar apuestas.
8. **Estadísticas y analítica (PDF pág. 7):** Almacenamiento masivo diario (millones de registros) de apuestas, montos, comportamiento, carreras, rendimiento histórico de medianos, probabilidades, tendencias y eventos. No requiere tiempo real estricto (retraso de minutos permitido; reportes premium hasta un día).
9. **Auditoría y control de fraude (PDF pág. 7-8):** Registro inalterable y conservado al menos 5 años de toda operación que afecte saldos, apuestas o resultados; consulta de solo lectura para auditor; detección de fraudes y reconstrucción exacta del reparto de premios.

---

3. Aclaración Metodológica y Aspectos Transversales

- **Diferenciación entre Capacidades Principales y Lista de Referencia para División de Servicios:**
  En la pág. 13 (Decisión 2), el PDF presenta una lista de capacidades como referencia para evaluar qué separar o agrupar en servicios: *(usuarios, medianos, carreras, apuestas, billetera, resultados, premios, notificaciones, estadísticas, auditoría)*. En dicha lista de trabajo se mencionan "resultados" y "premios" como conceptos separados, pero en el cuerpo principal del enunciado funcional (pág. 6) ambas forman una única capacidad denominada **"6. Resultados y premios"**. El diseño arquitectónico respetará las 9 capacidades del dominio de negocio y analizará su división técnica en P1-2.
- **Aspectos Transversales de Arquitectura (No capacidades de negocio aisladas):**
- **Seguridad (PDF pág. 12):** Autenticación, autorización RBAC, protección de APIs, cifrado en tránsito/reposo, credenciales, rate limiting, anti-bot y segmentación de red pública vs. privada.
- **Costos y Simplicidad (PDF pág. 12-13, 19, 20):** La directriz de Dyllan ("no pagar infraestructura de Netflix para una carrera de cinco medianos") es un criterio rector de gobernanza y economía técnica que restringe la sobreingeniería en todo el diseño.

---

4. Regla Estricta del Cierre de Apuestas

- **Independencia de Intervención Humana:**
  El cierre de apuestas está fijado exactamente a la hora de inicio de la carrera según el **reloj del servidor** (PDF pág. 3). **No depende de una orden manual ni de un botón accionado por el Operador de Carrera**. Cualquier solicitud recibida tras la marca de tiempo de cierre del servidor es rechazada irrevocablemente, independientemente de la latencia del cliente o de cuándo fue despachada por el usuario.

P1-1.2: REGLAS DEL NEGOCIO Y CICLO DE CARRERA

1. Modelo de Premios: Apuesta Mutua (Pari-Mutuel) (PDF pág. 3)

El sistema opera bajo un esquema estricto de bolsa común (*Pari-Mutuel*): - **Bolsa Bruta (B):** Sumatoria total de los montos apostados por todos los usuarios en la carrera c :

$$B = \sum_{i=1}^{N} \text{monto}_i$$

- **Retención de la Casa (R):** La casa (Salchipapa Maker Racing S.A.S.) retiene exactamente el **10%** de la bolsa total:

$$R = 0.10 \times B$$

- **Bolsa Neta Distribuible (B_{neta}):** Fondo total destinado a premiar a los apostadores ganadores (**90%** de la bolsa):

$$B_{\text{neta}} = 0.90 \times B$$

- **Reparto Proporcional a Ganadores:** Si el mediano ganador es M_w , el conjunto de apuestas ganadoras es $G = \{a \mid a.\text{mediano} = M_w\}$. La masa total apostada al ganador es:

$$T_w = \sum_{a \in G} a.\text{monto}$$

El premio asignado a una apuesta individual ganadora $a_k \in G$ se calcula en estricta proporción a su contribución individual:

$$\text{Premio}(a_k) = B_{\text{neta}} \times \left(\frac{a_k.\text{monto}}{T_w} \right) = \frac{0.90 \times B}{T_w} \times a_k.\text{monto}$$

- **Fijación de la Cuota Definitiva:** La cuota final oficial por unidad apostada (Q_w) queda sellada matemáticamente en el instante en que cesan las apuestas:

$$Q_w = \frac{B_{\text{neta}}}{T_w} = \frac{0.90 \times B}{T_w}$$

Por esta razón, **el premio final solo se conoce cuando se cierran las apuestas** y se certifica el ganador oficial (PDF pág. 3).

2. Cuotas Estimadas en Vivo (PDF pág. 3)

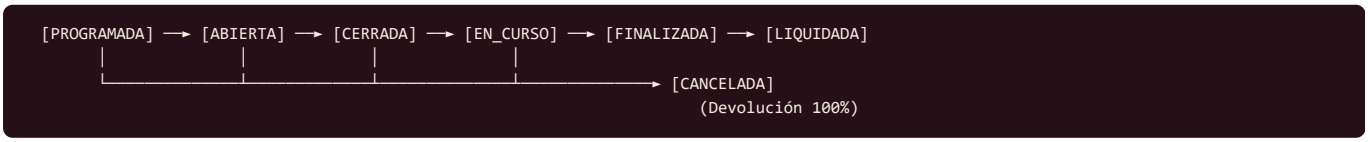
- Mientras la carrera permanece en estado $\texttt{ABIERTA}$, la plataforma calcula y expone a los apostadores una **cuota estimada** para cada mediano m :

$$Q_{\text{estimada}}(m) = \frac{0.90 \times B_{\text{actual}}}{T_m}$$

donde T_m es el total acumulado sobre el mediano m hasta ese instante. - Estas cuotas varían dinámicamente con cada nueva apuesta que ingresa. - **Naturaleza informativa:** La cuota mostrada es estrictamente una estimación proyectada; no constituye una cuota fija garantizada al momento de apostar.

3. Ciclo de Vida Formal de una Carrera (PDF pág. 3)

El PDF define formalmente una máquina de estados determinista para cada carrera:



Transiciones y Efectos de Estado:

1. `PROGRAMADA`: Carrera configurada con fecha, hora, pista, distancia y participantes. No admite apuestas.
2. `ABIERTA`: Ventana formal de admisión de apuestas. Se calculan cuotas estimadas.
3. `CERRADA`: Cese estricto e irrevocable de recepción de apuestas al llegar la hora de inicio (reloj del servidor). Bolsa común fijada.
4. `EN_CURSO`: Competencia deportiva en desarrollo. Las apuestas permanecen congeladas y selladas.
5. `FINALIZADA`: Conclusión de la carrera en pista. El sistema queda a la espera de la recepción del resultado oficial verificado.
6. `LIQUIDADA`: Resultado oficial procesado, ganadores identificados, premios calculados y acreditados en las billeteras de los usuarios.
7. `CANCELADA`: Transición de excepción posible **desde cualquier estado anterior a** `FINALIZADA`. Provoca la devolución íntegra (100%) de los montos apostados a todos los usuarios participantes.

4. Regla de Múltiples Apuestas por Usuario (PDF pág. 3)

- Un usuario tiene autorización expresa de emitir múltiples apuestas en la misma carrera.
- Puede apostar varias veces por el mismo mediano (ej. dos apuestas de 100 EIA Coins separadas) o diversificar apostando a distintos medianos.
- **Distinción arquitectónica fundamental:** Un nuevo envío de apuesta con un nuevo identificador de intención constituye una operación de negocio legítima y debe ser procesada. La deduplicación debe actuar estrictamente sobre reintentos de una misma transacción (`idempotency_key` u `operation_id`).

P1-1.3: PARÁMETROS CRÍTICOS DE CARGA, LATENCIA, DURABILIDAD, LIQUIDACIÓN Y RETENCIÓN

Con base en la sección "*El problema de las cargas masivas*" (PDF pág. 8) y la tabla de "*Supuestos de diseño*" (PDF págs. 8-9), se consolidan los parámetros cuantitativos obligatorios del sistema:

Parámetro / Dimensión	Criterio Cuantitativo Explícito	Requisito Fuente	Implicación de Diseño Arquitectónico
Usuarios Concurrentes (Normal)	500 usuarios conectados simultáneamente	PDF pág. 8	Carga base manejable con recursos mínimos y configuración liviana.
Usuarios Concurrentes (Pico Máximo)	100.000 usuarios conectados en la final de la Copa Intersemestral	PDF pág. 8	Requiere escalamiento horizontal elástico en capa de entrada y lectura eficiente.
Pico de Transacciones de Apuesta	**10.000 apuestas por segundo** sostenidas durante los 30 segundos previos al cierre	PDF pág. 8	Carga transaccional crítica: 300.000 apuestas en 30 segundos. Demanda bufferizado, procesamiento eficiente y particionamiento.
Latencia de Respuesta al Apostar	**< 1 segundo** para notificar al usuario si la apuesta fue aceptada o rechazada	PDF pág. 9	Límite estricto de latencia percibida por el usuario. Prohíbe arquitecturas de espera indefinida.
Durabilidad y Pérdida de Datos	**Cero tolerancia a pérdida:** ninguna apuesta aceptada ni movimiento de billetera puede perderse	PDF pág. 9	Persistencia con confirmación atómica y write-ahead log o almacenamiento con réplicas durables.
Tiempo Máximo de Liquidación	**< 15 minutos** para pagar todos los premios tras la publicación del resultado oficial	PDF pág. 9	Liquidación de más de 1.000.000 de apuestas (PDF pág. 6) distribuida mediante procesamiento por lotes/paralelo; prohíbe peticiones síncronas HTTP masivas monolíticas.
Frescura de Estadísticas	Métricas operativas con tolerancia de minutos de retraso; reportes premium hasta un día	PDF pág. 9	Desacoplamiento analítico: permite procesamiento en streaming/batch fuera de la ruta crítica transaccional.
Retención de Auditoría	Conservación inalterable durante **al menos 5 años**	PDF pág. 9	Almacenamiento histórico inmutable, optimizado para consulta de solo lectura por auditores.

P1-1.4: INVARIANTES DEL SISTEMA (SALDO, DUPLICACIÓN, CIERRE Y AUDITORÍA)

Los invariantes son condiciones formales de corrección que el sistema jamás puede violar bajo ninguna circunstancia de carga, concurrencia o fallo:

1. **Invariante de Moneda y No Doble Gasto (PDF pág. 6, 15):** - *Regla:* Un EIA Coin no puede ser gastado dos veces. - *Invariante:* $\text{Saldo}(u) \ge 0 \quad \forall u \in \text{Usuarios}$. El saldo contable de un usuario nunca puede ser negativo, aun cuando envíe múltiples apuestas en paralelo desde pestañas o conexiones simultáneas.
2. **Invariante de Cierre Temporal Estricto (PDF pág. 3, 5, 8, 9, 15):** - *Regla:* Cierre según reloj del servidor a la hora programada T_{inicio}. - *Invariante:* Si $t_{\text{recepción}}(\text{apuesta}) \ge T_{\text{inicio}}$, entonces $\text{Estado}(\text{apuesta}) = \text{RECHAZADA}$. Ninguna apuesta puede ser admitida en la bolsa mutua después del corte del servidor, sin importar cuándo la despachó el cliente.
3. **Invariante de Integridad Pari-Mutuel (PDF pág. 3):** - *Regla:* Conservación matemática estricta de la bolsa. - *Invariante:* $\sum \text{Premios Acreditados} + \text{Comisión Casa (10\%)} = B_{\text{total}}$. El sistema no puede emitir premios que superen el fondo recaudado ni retener más del porcentaje establecido.
4. **Invariante de No Duplicación por Idempotencia (PDF pág. 9, 15):** - *Regla:* Pulsaciones múltiples ("APOSTAR" repetido 4 veces) no multiplican el débito ni la apuesta. - *Invariante:* $\forall \text{operación } } op$, el efecto sobre la billetera y sobre las apuestas aceptadas debe ocurrir exactamente una vez para una misma clave lógica de idempotencia:

$$f(f(op)) = f(op)$$

5. **Invariante de Registro Inmutable y Solo Lectura en Auditoría (PDF pág. 7, 8):** - *Regla:* Trazabilidad absoluta de eventos contables y de resultados. - *Invariante:* Toda mutación sobre saldos, creación de apuestas, cambios de estado de carreras o resultados genera un registro inmutable. - *Aclaración conceptual:*
   - **Append-only:** Propiedad estructural del modelo de datos donde se permite insertar nuevos registros cronológicos de eventos sin modificar ni sobrescribir los registros preexistentes.
   - **Solo lectura:** Política de permisos y control de acceso asignada al rol Auditor en el plano de autorización.
   - La protección física y lógica efectiva contra alteraciones (como firmas criptográficas, inmutabilidad a nivel de almacenamiento o políticas WORM) se diseñará y justificará formalmente en las etapas de persistencia y seguridad.

P1-1.5: IDENTIFICACIÓN DE AMBIGÜEDADES, PREGUNTAS ABIERTAS Y RIESGOS TÉCNICOS

Para preservar la pureza metodológica, se formaliza la clasificación de elementos no definidos en el PDF:

Código	Categoría	Descripción	Impacto en el Diseño	Propuesta o Estrategia de Resolución
P1-PA-001	Pregunta Abierta	**Política de Redondeo y Remanentes de la Bolsa:** Al dividir $0.90 \times B$ entre las apuestas ganadoras, pueden generarse fracciones indivisibles de EIA Coins.	Posible descuadre contable si no se especifica el manejo de decimales.	**Propuesta técnica para P1-5:** Redondeo al entero inferior (*floor*) de cada premio individual y acumulación del residuo fraccionario en la cuenta de la casa o fondo de reserva del torneo.
P1-PA-002	Pregunta Abierta	**Escenario sin Ganadores:** El PDF no estipula el destino del 90% de la bolsa si ningún apostador eligió al mediano que cruzó la meta en primer lugar.	Vacío de negocio sobre retención o devolución.	**Propuesta técnica para P1-5:** Devolución íntegra (100%) a los apostadores participantes en esa carrera o acumulación (*rollover*) para la siguiente carrera. Se presentará a elección del usuario en P1-5.
P1-RIE-001	Riesgo / Desafío Técnico	**Tensión Latencia vs Concurrencia Extrema:** Responder en menos de 1 segundo mientras ingresan 10.000 apuestas/s en 30 segundos.	Riesgo de contención en base de datos al validar saldo y debitar bajo concurrencia.	Mitigación en P1-3, P1-5 y P1-6 mediante particionamiento por usuario, estructuras en memoria para validación rápida y aislamiento de colas.

Código	Categoría	Descripción	Impacto en el Diseño	Propuesta o Estrategia de Resolución
P1-RIE-002	Riesgo de Conectividad	**Incertidumbre de Red en el Cliente:** Un timeout de red no significa que la apuesta no fue procesada.	Riesgo de que el usuario reintente creyendo que falló.	Mitigación mediante `idempotency-key` obligatoria generada por el cliente antes de despachar la solicitud.

ETAPA P1-2: ARQUITECTURA Y COMPONENTES

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-003`
Versión: `v1.0`
Estado: `[x]` Consolidado (Estilo arquitectónico aprobado por el usuario; desglose de límites y componentes verificado por revisión autónoma)

P1-2.1 Y P1-2.2: DECISIÓN 1 — ESTILO ARQUITECTÓNICO APROBADO

1. Dirección Arquitectónica Aprobada: Arquitectura Híbrida

Se adopta formalmente una **Arquitectura Híbrida** estructurada a partir de un **Núcleo Modular Transaccional (Core Modular)** complementado con **Componentes y Workers Asíncronos Desacoplados**.

> **Fundamento metodológico y precisión de garantías:**
> La elección de este estilo **no garantiza por sí sola** el cumplimiento de la respuesta en menos de un segundo, la absorción de 10.000 apuestas/s ni el cierre exacto. Su mérito radica en que **elimina dependencias de red distribuidas intermedias en la ruta crítica** y simplifica el plano de consistencia al permitir transacciones locales para el débito y la apuesta, facilitando el diseño de los mecanismos específicos de concurrencia, particionamiento, durabilidad e idempotencia que se validarán en las etapas posteriores.

2. Estructura General del Estilo Aprobado

1. **Núcleo Transaccional Modular (Core Monolith replicable):** - Agrupa con límites estrictos de módulo: *Usuarios/Autenticación*, *Medianos*, *Carreras*, *Apuestas* y *Billetera de EIA Coins*. - **Frontera transaccional estricta:** La verificación de saldo, la deducción de EIA Coins y la persistencia de la apuesta admitida se diseñan para ejecutarse dentro de una **misma transacción local atómica**, suprimiendo la necesidad de protocolos de coordinación distribuida (2PC o Sagas con compensación) en la ruta crítica de admisión.
2. **Componentes y Workers Desacoplados de la Ruta Crítica:** - **Servicio Autónomo de Notificaciones:** Desacoplado físicamente para que cualquier fallo o lentitud en canales externos (push, correo) jamás bloquee el flujo de admisión de apuestas. - **Pipeline y Servicio de Analítica / Estadísticas:** Desacoplado del motor transaccional, alimentado por eventos para no canibalizar recursos de base de datos durante las consultas analíticas pesadas. - **Proceso / Worker de Liquidación Masiva y Devoluciones:** Proceso especializado que procesa el cálculo y dispersión masiva de premios fuera del hilo interactivo de los usuarios.

3. Justificación del Descarte de Alternativas (Obligatorio PDF pág. 13)

A. Descarte de Microservicios Puros

- **Justificación técnica:** Una descomposición en microservicios independientes con bases de datos privadas para cada capacidad introduce una sobrecarga significativa de coordinación distribuida. Exigiría implementar orquestación o coreografía de Sagas y transacciones compensatorias complejas únicamente para sincronizar el débito en billetera con el registro de la apuesta, incrementando el riesgo de condiciones de carrera y exposición a saldo negativo bajo concurrencia masiva.
- **Criterio de gobernanza y costo:** La complejidad operativa (service discovery, distributed tracing, mTLS, múltiples esquemas de despliegue y particionamiento) resulta desproporcionada frente a las necesidades operativas de Salchipapa Maker Racing S.A.S., contraviniendo la regla rectora de Dyllan sobre evitar costos y complejidad innecesarios (PDF pág. 12, 19, 20). *(Nota: El descarte se fundamenta en la complejidad de consistencia, operación y costo, no en una incapacidad intrínseca de rendimiento).*

B. Descarte del Monolito Tradicional (Acoplado / No modular)

- **Justificación técnica:** Un monolito convencional sin fronteras modulares estrictas propaga fallos entre dominios dispares: una degradación en el despacho de notificaciones externas puede agotar el pool de hilos o conexiones HTTP, bloqueando la recepción de apuestas legítimas.
- **Contención de recursos:** La ingesta diaria de millones de registros analíticos y la ejecución de consultas pesadas de tendencias competirían directamente por los mismos recursos de memoria y bloqueos de almacenamiento que el pico de apuestas previo al cierre.

P1-2.3: DEFINICIÓN DE RESPONSABILIDADES, LÍMITES Y UBICACIÓN DE CAPACIDADES

A continuación se formaliza la ubicación arquitectónica de las nueve capacidades del negocio, integrando expresamente los dominios de **Resultados**, **Liquidación de Premios**, **Devoluciones** y **Auditoría / Control de Fraude**:



Tabla de Ubicación y Frontera Transaccional de Responsabilidades

Capacidad / Responsabilidad	Módulo Lógico	Unidad Desplegable	Frontera Transaccional	Justificación de Ubicación
Usuarios y Autenticación	`UsersModule`	Núcleo Modular (Core)	Dentro de frontera	Autenticación y validación de permisos de sesión en memoria/borde sin llamadas remotas.
Medianos	`RacersModule`	Núcleo Modular (Core)	Lectura local / Caché	Datos maestros de baja volatilidad consultados directamente al validar la apuesta.

Capacidad / Responsabilidad	Módulo Lógico	Unidad Desplegable	Frontera Transaccional	Justificación de Ubicación
Carreras (Programación y Cierre)	`RacesModule`	Núcleo Modular (Core)	Dentro de frontera	El control del reloj del servidor y el estado de la carrera (`ABIERTA` / `CERRADA`) deben validarse atómicamente al recibir la apuesta.
Apuestas (Admisión e Idempotencia)	`BetsModule`	Núcleo Modular (Core)	**Ruta crítica transaccional**	Debe coordinarse atómicamente con la billetera para admitir la jugada.
Billetera de EIA Coins (Débito)	`WalletModule`	Núcleo Modular (Core)	**Ruta crítica transaccional**	Descuento atómico junto a la apuesta para garantizar que no exista saldo negativo.
Recepción de Resultado Oficial	`RacesModule`	Núcleo Modular (Core)	Transaccional administrativa	Sellado formal del ganador oficial por el Operador, disparando el evento de cierre de ciclo.
Liquidación de Premios	`SettlementModule`	Worker de Liquidación	**Fuera de la ruta crítica**	El cálculo sobre más de 1.000.000 de apuestas y la acreditación masiva se ejecutan asíncronamente en lotes para no bloquear peticiones HTTP.
Devoluciones por Cancelación	`SettlementModule`	Worker de Liquidación	**Fuera de la ruta crítica**	Si la carrera pasa a `CANCELADA` , un proceso batch dispersa el 100% de los reembolsos a las billeteras de manera controlada.
Notificaciones	`NotificationService`	Servicio Autónomo	**Fuera de la ruta crítica**	Su fallo no debe afectar el registro transaccional; consume eventos de dominio mediante colas.
Estadísticas y Analítica	`AnalyticsService`	Servicio Autónomo	**Fuera de la ruta crítica**	Procesa millones de eventos diarios en un almacén optimizado para lecturas agregadas sin sobrecargar la base de datos operativa.
Auditoría y Control de Fraude	`AuditModule`	Núcleo Core + Almacén Inmutable	Híbrida (Registro local + Análisis asíncrono)	Las mutaciones generan un registro de auditoría append-only atómico; el análisis forense y la consulta de solo lectura se ejecutan en réplica dedicada.

P1-2.4: DISTINCIÓN ENTRE MÓDULOS, UNIDADES DESPLEGABLES Y PROPIEDAD DE DATOS

Para evitar acoplamientos ocultos y respetar la regla de Dyllan, se formalizan las tres dimensiones arquitectónicas:

1. **Módulos Lógicos (Estructura de Código):**
   Encapsulan lógica de negocio, entidades y reglas en paquetes con interfaces explícitas. Un módulo no puede consultar directamente las tablas internas de otro módulo ni mutar sus estructuras en memoria; toda interacción interna se realiza mediante contratos de interfaz de dominio.
2. **Unidades Desplegables (Procesos en Ejecución):** - **`core-api`:** Servicio web/API escalable que contiene el Núcleo Modular Transaccional. Recibe el tráfico de apostadores y operadores. - **`settlement-worker`:** Proceso en segundo plano dedicado a la liquidación masiva de premios y dispersión de devoluciones. - **`notification-service`:** Demonio/consumidor independiente para despacho de mensajes multicanal. - **`analytics-service`:** Pipeline y API de consulta analítica especializada.
3. **Propiedad y Almacenamiento de Datos:** - **Base de Datos Operativa Transaccional (OLTP):** Propiedad exclusiva del Núcleo Modular. Aloja esquemas aislados para usuarios, carreras, apuestas, billetera y libro contable. Soporta transacciones ACID locales. - **Almacén Analítico e Histórico (OLAP):** Propiedad exclusiva del servicio de analítica. Esquema desnormalizado optimizado para consultas agregadas masivas y retención histórica de 5 años. - **Aislamiento de Recursos:** El almacén analítico se encuentra físicamente separado del almacén transaccional operativo, impidiendo que la generación de reportes premium degrade el rendimiento de las apuestas.

P1-2.5: ESTRATEGIA DE ESCALAMIENTO INDEPENDIENTE Y MITIGACIÓN DE CUELLOS DE BOTELLA

1. Escalamiento Horizontal del Núcleo Modular (`core-api`)

- El Núcleo Modular se despliega en **múltiples instancias idénticas** detrás de un balanceador de carga con terminación TLS.
- Las instancias son completamente *stateless* a nivel de sesión HTTP (autenticación mediante tokens criptográficos verificables localmente sin consulta de sesión a base de datos).

2. Cuellos de Botella del Almacenamiento Compartido y Mitigaciones

- **Desafío:** Al tener múltiples instancias de `core-api` concurriendo sobre la misma base de datos operativa durante el pico de 10.000 apuestas/s, el cuello de botella físico se traslada a los bloqueos de fila de saldo de billetera y contención en la tabla de apuestas.
- **Estrategia de mitigación arquitectónica:** 1. *Particionamiento lógico por usuario:* Las actualizaciones de saldo se bloquean estrictamente a nivel del registro individual de cada usuario (`SELECT FOR UPDATE` sobre la fila del usuario), evitando bloqueos de tabla. Como un usuario emite un número finito de apuestas, la contención ocurre por usuario y no globalmente. 2. *Separación de lecturas y escrituras:* Las consultas masivas de cuotas estimadas, catálogo de medianos y perfiles de carreras son atendidas mediante réplicas de lectura y mecanismos de almacenamiento en caché en memoria, reservando el nodo primario de base de datos exclusivamente para inserciones de apuestas y débitos. 3. *Aislamiento del Worker de Liquidación:* Durante la liquidación masiva posterior al cierre, el worker procesa las acreditaciones en bloques (*chunks*) con límites de concurrencia para no saturar las conexiones ni bloquear la base de datos transaccional si hay otra carrera abierta en paralelo.

ETAPA P1-3: FLUJO CRÍTICO Y COMUNICACIÓN SÍNCRONA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-004`
Versión: `v1.0`
Estado: `[x]` Consolidado y verificado por revisión autónoma

P1-3.1: FLUJO COMPLETO DE ACEPTACIÓN / RECHAZO DE UNA APUESTA

El flujo de procesamiento de una apuesta representa la ruta transaccional más crítica del sistema, sujeta a la restricción de respuesta en menos de 1 segundo (PDF pág. 9, 15).



Detalle Secuencial Paso a Paso:

1. **Despacho del Cliente:** El apostador envía una petición HTTP `POST /api/v1/bets` transportando en el cuerpo: identificador de carrera (`race_id`), mediano seleccionado (`racer_id`), monto a apostar (`amount`) y en la cabecera HTTP un identificador único de intención (`Idempotency-Key` generado por el cliente).
2. **Filtrado en Borde (API Gateway):** Aplica *rate limiting* por IP/usuario y validación de firma criptográfica del token JWT. Si la tasa es excedida, rechaza inmediatamente con `HTTP 429 Too Many Requests` sin tocar el núcleo transaccional.
3. **Apertura de Transacción Local (ACID):** La instancia de `core-api` abre una transacción atómica local en la base de datos relacional operativa.
4. **Verificación de Idempotencia:** Se comprueba si la `idempotency_key` ya fue procesada previamente para este usuario. Si ya existe, se retorna el resultado previamente registrado (`HTTP 200 OK`) sin duplicar débitos.

5. **Comprobación de Reglas de Admisión y Cierre:** - Comprueba existencia y habilitación del usuario. - Comprueba que el mediano participe efectivamente en dicha carrera. - **Regla estricta de cierre:** Se evalúa de forma atómica: `server_current_timestamp < race.start_time` y `race.status == 'ABIERTA'`. Si la hora del servidor alcanzó o superó el inicio de la carrera, la transacción se aborta y se responde `HTTP 409 Conflict / REJECTED_RACE_CLOSED`.
6. **Bloqueo Pesimista de Billetera:** Se ejecuta `SELECT balance FROM wallets WHERE user_id = :uid FOR UPDATE`. Este bloqueo a nivel de fila garantiza que si el usuario emite dos apuestas simultáneas desde dos dispositivos distintos, se procesen de forma estrictamente secuencial.
7. **Validación de Saldo y Ejecución Atómica:** - Si `balance < amount`, la transacción se revierte (`ROLLBACK`) y se responde `HTTP 422 Unprocessable Entity / INSUFFICIENT_FUNDS`. - Si `balance >= amount`, se actualiza el saldo contable (`balance = balance - amount`), se inserta el registro de la apuesta en la tabla `bets`, y se inserta un registro en la tabla transaccional `outbox` con el evento `BetPlaced` para su posterior difusión asíncrona.
8. **Commit Local:** Se confirma la transacción atómica en la base de datos (`COMMIT`).
9. **Confirmación Determinista al Apostador:** El servidor retorna `HTTP 201 Created` con el detalle de la apuesta aceptada y el saldo restante actualizado, completando el ciclo en decenas de milisegundos (holgadamente dentro del límite de 1 segundo).

P1-3.2: COHERENCIA ESTRICTA ENTRE APUESTA Y DESCUENTO EN BILLETERA

Al haber adoptado la Arquitectura Híbrida (`P1-DEC-001`), la coherencia entre la apuesta y el descuento se resuelve mediante **atomicidad transaccional local** en lugar de transacciones distribuidas: - **Todo o Nada (All-or-Nothing):** No existe ninguna ventana de vulnerabilidad donde la apuesta quede registrada pero el dinero no se descuente, ni donde el saldo se reduzca pero la apuesta falle. - **Invariante Preservado:** La integridad contable queda garantizada por las propiedades ACID del motor de base de datos relacional. La propagación hacia componentes externos (analítica, notificaciones, cálculo de cuotas dinámicas) se realiza de forma asíncrona a través del patrón **Transactional Outbox**, garantizando que un evento hacia el broker solo se despache si la transacción local de BD hizo `COMMIT` exitoso.

P1-3.3: VALIDACIONES Y MECANISMO DE CIERRE ESTRICTO

1. Definición Formal del Momento de Aceptación

Una apuesta **solo se considera aceptada por el sistema** en el instante exacto en que la transacción local de base de datos ejecuta un `COMMIT` exitoso. Si en cualquier punto previo al commit la transacción falla, se revierte o supera la hora de corte, la apuesta se considera **rechazada de forma determinista**, sin que se registre la jugada ni se debite el saldo.

2. Tratamiento de Esperas por Bloqueos y Concurrencia al Cierre

- **El Escenario Crítico:** Un usuario despacha una solicitud de apuesta a las `11:59:59.950` para una carrera que inicia a las `12:00:00.000`. Al ingresar a la base de datos, la solicitud queda en espera (*lock wait*) debido a que otra transacción concurrente sobre el mismo usuario está ejecutándose. Para cuando la transacción adquiere el bloqueo de fila, el reloj del servidor marca las `12:00:00.050`.
- **Regla Inflexible de Rechazo:** Aunque la petición HTTP haya llegado a la infraestructura antes del corte, **si al momento de evaluar la sentencia atómica en el motor la hora del servidor alcanzó o superó `start_time`, la apuesta es irrevocablemente rechazada**.
- **Mecanismo Atómico Conjunto (Hora + Estado + Saldo):** La validación no se delega a una lectura aislada previa de `CLOCK_TIMESTAMP()`. Se ejecuta como condición de inserción y bloqueo dentro de la misma sentencia: `sql -- Inserción atómica condicionada al reloj exacto y estado de carrera INSERT INTO bets (id, user_id, race_id, racer_id, amount, created_at) SELECT :bet_id, :user_id, :race_id, :racer_id, :amount, CLOCK_TIMESTAMP() FROM races WHERE races.id = :race_id AND races.status = 'ABIERTA' AND CLOCK_TIMESTAMP() < races.start_time;` Si la carrera transicionó a `CANCELADA` o `CERRADA`, o si la espera por bloqueos hizo que `CLOCK_TIMESTAMP() >= start_time`, la sentencia afecta exactamente **0 filas**. El backend detecta `rows_affected == 0`, ejecuta `ROLLBACK` de la transacción y retorna de inmediato `HTTP 409 Conflict`

`(RACE_CLOSED_OR_NOT_OPEN)` al cliente. Esto garantiza que ninguna espera por contención degrade la regla del corte del servidor.

P1-3.4: TABLA DE COMUNICACIONES SÍNCRONAS EXIGIDA (PDF pág. 14)

Conforme a la estructura obligatoria requerida en la Decisión 3 del PDF, se identifican las comunicaciones que operan de forma estrictamente síncrona:

Servicio Origen	Servicio Destino	Protocolo	Información Solicitada	¿Por qué debe ser síncrona?	Estrategia ante Fallo / Timeout
Cliente Apostador (Web/App)	**API Gateway / Edge**	HTTPS / REST (JSON)	Solicitud de autenticación, consulta de carreras y emisión de apuesta (`POST /bets`).	El usuario requiere retroalimentación interactiva inmediata en pantalla (< 1 s) para saber si su apuesta fue aceptada.	Timeout de cliente a los 800 ms. Si no hay respuesta, el cliente puede reintentar de forma segura utilizando la misma `Idempotency-Key` .
API Gateway	**Núcleo Modular (`core-api`)**	HTTP/2 o gRPC interno	Enrutamiento de peticiones de validación y emisión de apuestas con token JWT validado.	Tráfico en la ruta crítica del usuario; requiere latencia sub-milisegunda entre capas de red privada.	Circuit breaker con timeout de 600 ms. En caso de timeout, el gateway responde `HTTP 504 Gateway Timeout` informando que el estado es incierto e invitando a reintentar con la misma clave.
Núcleo Modular (`core-api`)	**Base de Datos Operativa (OLTP)**	Protocolo nativo de BD (TCP/Pool de conexiones)	Transacción ACID: `SELECT FOR UPDATE` de billetera, validación de corte de carrera e `INSERT` de apuesta.	Garantía obligatoria de no doble gasto, verificación estricta de saldo e integridad contable sin condiciones de carrera.	Timeout transaccional de 500 ms. Si la BD no responde, se ejecuta rollback automático y se retorna error temporal `HTTP 503 Service Unavailable` .
Cliente Operador	**Núcleo Modular (`core-api`)**	HTTPS / REST	Publicación del resultado oficial verificado de una carrera (`POST /races/{id}/results`).	El operador debe recibir confirmación inmediata del sellado del resultado para dar paso al proceso de liquidación.	Timeout de 2 s. Reintentos manuales idempotentes basados en el identificador único de la carrera.
Cliente Auditor	**Núcleo Modular / Réplica de Auditoría**	HTTPS / REST	Consulta de registros inalterables de apuestas, saldos y reconstrucción matemática de premios.	Operación de consulta interactiva para tareas de fiscalización y auditoría forense.	Timeout de 5 s. Lectura dirigida a réplica de lectura para no sobrecargar el nodo transaccional primario.

P1-3.5: GESTIÓN DE TIMEOUTS, REINTENTOS Y RESPUESTAS INCIERTAS

En sistemas transaccionales distribuidos de apuestas bajo alta concurrencia, es imperativo distinguir tres estados lógicos de una operación desde la perspectiva del cliente: 1. **`ACEPTADA` (Determinista):** Transacción local confirmada (`COMMIT`), apuesta persistida en la bolsa y saldo descontado. El cliente recibe `HTTP 201 Created` . 2. **`RECHAZADA` (Determinista):** Regla de negocio no satisfecha (saldo insuficiente, mediano no participante o llegada posterior a la hora de cierre según el reloj del servidor). El cliente recibe `HTTP 422 Unprocessable Entity` o `HTTP 409 Conflict` . 3. **`DESCONOCIDA / INCIERTA` (Por incertidumbre de red o timeout técnico):** La petición no completó su ciclo de respuesta antes del límite de espera técnico del cliente o del gateway (configurado preventivamente en ~800 ms para no retener conexiones bloqueadas).

> **Aclaración crítica sobre latencia:**
> El timeout de cliente de 800 ms **no constituye una garantía de que la operación de negocio se resolvió con éxito en menos de un segundo**. Es un mecanismo defensivo para liberar el socket ante fallos de conectividad. Cuando ocurre un timeout, el estado de la apuesta permanece estrictamente en condición de incertidumbre.

Protocolo de Resolución de Incertidumbre y Reintentos:

1. **Incertidumbre de Red en el Cliente:**
   Si el cliente web/móvil experimenta timeout o pérdida de paquete antes de recibir confirmación, **la aplicación no debe asumir que la apuesta falló ni que fue cobrada**.
2. **Reintentos Idempotentes Seguros:**
   La aplicación cliente retiene en memoria la misma `Idempotency-Key` (UUIDv4) generada originalmente. Al ejecutar un reintento automático o tras la acción del usuario, reenvía la misma clave. El servidor detectará si la transacción ya fue confirmada (retornando el estado final sin duplicar cobros) o si se encuentra en procesamiento.
3. **Consulta de Estado Alternativa:**
   Si persiste la incertidumbre, el cliente puede invocar el endpoint de lectura liviana `GET /api/v1/bets/by-key/{idempotency_key}` para inspeccionar el estado definitivo registrado en el servidor antes de permitir al usuario formular una nueva intención de apuesta.

ETAPA P1-4: MENSAJERÍA Y COMUNICACIÓN ASÍNCRONA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-005`
Versión: `v1.0`
Estado: `[x]` Consolidado (Elección de Apache Kafka aprobada por el usuario en P1-DEC-002; diseño de eventos, topología y outbox verificado por revisión autónoma)

P1-4.1 Y P1-4.2: DECISIÓN 5 — ELECCIÓN DE LA TECNOLOGÍA DE MENSAJERÍA

1. Dirección Aprobada: Apache Kafka como Broker Único del Sistema

Se adopta formalmente **Apache Kafka** como plataforma exclusiva de mensajería y streaming de eventos para la Liga de Apuestas de Medianos.

2. Justificación Técnica y Comparativa Frente al Dominio

A. Ajuste al Patrón de Múltiples Consumidores Independientes y Eventos Retenidos

- **Naturaleza del Dominio:** En la plataforma, un único evento de negocio (por ejemplo, la admisión de una apuesta o la publicación del resultado oficial) interesa concurrentemente a múltiples subsistemas:
- Motor de estimación de cuotas dinámicas en vivo.
- Servicio de auditoría y control de anomalías.
- Pipeline de estadísticas y analítica para reportes diarios y venta premium.
- Servicio de notificaciones multicanal hacia los apostadores.
- **Ventaja de Diseño de Kafka:** En Kafka, el almacenamiento se estructura como un log distribuido de confirmación en disco (*commit log* append-only). Múltiples grupos de consumidores (`Consumer Groups`) leen concurrentemente la misma partición a su propio ritmo sin necesidad de duplicar el mensaje físicamente en almacenamiento ni generar colas aisladas por cada suscriptor. Cada grupo mantiene su propio cursor o puntero de lectura (`offset`).

B. Justificación del Descarte de Alternativas

1. **Descarte de RabbitMQ como broker principal del diseño:** - Si bien RabbitMQ soporta patrones avanzados de enrutamiento AMQP, colas tradicionales y extensiones como *RabbitMQ Streams* (que ofrecen semánticas de log secuencial), el diseño del sistema encaja de forma más directa y estándar en el modelo de tópicos particionados y grupos de consumidores de Kafka, donde los eventos retenidos por ventana temporal permiten desacoplar de forma nativa a consumidores con velocidades de procesamiento dispares (ej. notificaciones lentas vs. cálculo ágil de cuotas). - *(Aclaración de descarte):* No se descarta a RabbitMQ por una supuesta incapacidad técnica ni por limitaciones genéricas de throughput, sino porque la semántica de publicación basada en log distribuido con particionamiento explícito por clave de negocio (`race_id`, `user_id`) satisface de manera más compacta y uniforme todos los flujos asíncronos del caso de estudio sin requerir tecnologías heterogéneas.
2. **Descarte de la Solución Dual (RabbitMQ + Kafka combinados):** - El PDF señala taxativamente (pág. 11, 15): "*No existe obligación de utilizar ambos... Si utilizan ambos, deberán justificar por qué mantener dos tecnologías diferentes representa una ventaja suficiente para asumir la complejidad adicional*". - Para el alcance de la Liga de Medianos, **no se ha demostrado una ventaja operativa suficiente que compense la complejidad de aprovisionar, monitorizar, asegurar y operar dos plataformas de mensajería distintas**. Una sola tecnología bien dimensionada cubre tanto el streaming de eventos como la distribución de tareas de liquidación.

C. Matización Rigurosa de Rendimiento y Rendimiento Asíncrono

- **Distinción entre Apuestas/s y Eventos/s:**
- El pico del negocio es de **10.000 apuestas por segundo** en la capa transaccional HTTP/BD durante 30 segundos.

- Cada apuesta aceptada produce un evento principal de dominio (`BetPlaced`). No obstante, si se emitieran eventos secundarios de auditoría o cuotas sin consolidación, la tasa de eventos podría multiplicarse. El diseño adopta **1 evento canónico por apuesta admitida**, resultando en una tasa asíncrona pico de ~10.000 eventos/s hacia Kafka.
- **Supuestos de Dimensionamiento:**
- Tamaño promedio de evento `BetPlaced` serializado en JSON/Avro: `\sim 500\text{ bytes}` .
- Tasa de datos en pico: `10.000\text{ ev/s} \times 500\text{ bytes} \approx 5\text{ MB/s}` de rendimiento de escritura sostenido durante 30 segundos (`\approx 150\text{ MB}` totales por ráfaga), lo cual es absorbible por un clúster moderado de Kafka con particiones adecuadas sin representar una carga insalvable.
- **Compromiso entre Paralelismo y Orden por Partición:**
- En Kafka, el orden estricto de entrega solo se garantiza **dentro de una misma partición**.
- Aumentar el número de particiones eleva el paralelismo de consumo, pero distribuye los eventos. Para preservar el orden causal donde es obligatorio (ej. ciclo de una carrera), se utiliza la clave de partición adecuada.

D. Delimitación Estricta entre Retención Operativa en Kafka y Auditoría Inalterable a 5 Años

- **Retención Operativa en Kafka:** Kafka se configura con una retención temporal acotada (ej. **7 días**). Su función es absorber ráfagas, tolerar caídas transitorias de consumidores y permitir reprocesamiento reciente (*replay* operativo). **Kafka no es el repositorio definitivo de auditoría inalterable a 5 años**.
- **Auditoría Legal y Forense a 5 Años:** Se garantiza mediante la ingesta continua de eventos desde Kafka hacia una base de datos o almacenamiento inmutable de auditoría (esquema append-only con controles de integridad criptográfica y permisos exclusivos de solo lectura para el auditor).
- **Protección contra Efectos Duplicados en Reprocesamiento (*Replay*):**
- Si un consumidor retrocede su offset para reprocesar eventos, **jamás debe re-acreditar premios ni emitir reembolsos duplicados**.
- Los consumidores de billetera y liquidación operan con **idempotencia estricta en la base de datos de destino** (ej. tablas contables con restricciones `UNIQUE (race_id, user_id, transaction_type)`), ignorando transacciones previamente ejecutadas.

P1-4.3: DEFINICIÓN CANÓNICA DE EVENTOS EN UN ÚNICO IDIOMA

Para dar cumplimiento formal al requisito del PDF (pág. 10, 14: "*Usen un único idioma para nombrarlos*"), se adopta **Español formal en formato PascalCase** en toda la plataforma:

1. `ApuestaAceptada` : Emitido cuando una apuesta es validada, descontada de la billetera y confirmada en la base de datos.
2. `CarreraIniciada` : Emitido cuando la carrera arranca oficialmente en pista y cesa irrevocablemente el corte.
3. `CarreraFinalizada` : Emitido cuando el último mediano cruza la meta y la carrera concluye en pista.
4. `ResultadoOficialPublicado` : Emitido cuando el Operador de Carrera registra y sella la foto de llegada y el ganador oficial.
5. `LotePremiosCalculado` : Emitido por el worker de liquidación detallando las apuestas premiadas por bloque.
6. `PremioAcreditado` : Emitido cada vez que se abona exitosamente la recompensa en la billetera de un apostador ganador.
7. `CarreraCancelada` : Emitido si la carrera se suspende antes de finalizar, activando el proceso masivo de devoluciones.
8. `DevolucionAcreditada` : Emitido al reintegrarse el 100% de los fondos de una apuesta tras cancelación.

P1-4.4: TABLA DE PRODUCTORES, CONSUMIDORES, TOPOLOGÍA Y CLAVES (PDF pág. 14)

Conforme a la estructura obligatoria de la Decisión 4 del PDF, se define el catálogo de eventos asíncronos:

Evento	Productor	Tecnología	Consumidor(es)	Clave de Partición ( )	Propósito / Efecto del Consumo
`ApuestaAceptada`	Núcleo Modular (`core-api` vía Outbox)	Apache Kafka (Topic: `apuestas.aceptadas`)	1. `CalculadorCuotas` 2. `ServicioAnalitica` 3. `ServicioAuditoria` 4. `ServicioNotificaciones`	`user_id` (o `bet_id`)	**Distribución de carga:** Se utiliza `user_id` para distribuir el pico de 10.000 apuestas/s uniformemente entre todas las particiones del tópico, garantizando paralelismo real de consumo en analítica y notificaciones. Los eventos de una misma carrera se agregan en el calculador de cuotas mediante ventanas en streaming.
`CarreraIniciada`	Núcleo Modular (`RacesModule` vía Outbox)	Apache Kafka (Topic: `carreras.eventos`)	1. `ServicioNotificaciones` 2. `ServicioAnalitica`	`race_id`	Notifica a usuarios en vivo y actualiza el estado de la transmisión en tiempo real.
`CarreraFinalizada`	Núcleo Modular (`RacesModule` vía Outbox)	Apache Kafka (Topic: `carreras.eventos`)	1. `ServicioNotificaciones` 2. `ConsolaOperador`	`race_id`	Alerta al operador y usuarios del fin de la carrera en pista a la espera de resultados oficiales.
`ResultadoOficialPublicado`	Núcleo Modular (`RacesModule` vía Outbox)	Apache Kafka (Topic: `carreras.resultados`)	1. `WorkerLiquidacion` 2. `ServicioNotificaciones` 3. `ServicioAuditoria`	`race_id`	**Disparador de liquidación:** El worker toma el resultado oficial y arranca el cálculo proporcional del 90% de la bolsa.
`LoteLiquidacionGenerado`	`WorkerLiquidacion`	Apache Kafka (Topic: `liquidacion.lotes`)	`WorkersDispersadoresSaldo` (Pool de hilos en `core-api` / worker)	`batch_id`	**Distribución de carga:** Divide 1M de apuestas en lotes particionados por `batch_id` para dispersar

Evento	Productor	Tecnología	Consumidor(es)	Clave de Partición ( )	Propósito / Efecto del Consumo
					el trabajo entre todos los workers y particiones.
`PremioAcreditado`	Worker / Módulo Billetera	Apache Kafka (Topic: `billetera.movimientos`)	1. `ServicioNotificaciones` 2. `ServicioAuditoria` 3. `ServicioAnalitica`	`user_id`	Notifica al usuario ganador ("¡Ganaste X EIA Coins!") y registra el movimiento contable en auditoría.
`CarreraCancelada`	Núcleo Modular (`RacesModule` vía Outbox)	Apache Kafka (Topic: `carreras.eventos`)	1. `WorkerLiquidacion` (Modo Reembolso) 2. `ServicioNotificaciones` 3. `ServicioAuditoria`	`race_id`	Dispara el proceso de reembolso del 100% de los montos a cada apostador participante.
`DevolucionAcreditada`	Worker / Módulo Billetera	Apache Kafka (Topic: `billetera.movimientos`)	1. `ServicioNotificaciones` 2. `ServicioAuditoria`	`user_id`	Notifica al apostador el reintegro de sus EIA Coins por cancelación de carrera.

P1-4.5: ESQUEMA Y CONTENIDO MÍNIMO DE LOS EVENTOS

Todos los eventos de la plataforma comparten una estructura base canónica con versionado semántico para garantizar evolución compatible de esquemas:

```
{
  "eventId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "eventType": "ApuestaAceptada",
  "schemaVersion": "1.0",
  "timestamp": "2026-10-27T18:30:00.125Z",
  "producer": "core-api/instance-03",
  "correlationId": "corr-89102-uuid",
  "data": {
    "betId": "bet-994821",
    "raceId": "race-copa-final-2026",
    "userId": "usr-48192",
    "racerId": "racer-frodito-turbo",
    "amount": 500,
    "userBalanceAfter": 1500,
    "idempotencyKey": "idem-client-8812739"
  }
}
```

Campos Mínimos Obligatorios:

- `eventId` : Identificador único global (UUIDv4) del evento para deduplicación en consumidores.
- `eventType` : Nombre canónico del evento en PascalCase.
- `schemaVersion` : Versión del contrato de datos para soportar evolución.
- `timestamp` : Marca temporal en UTC generada por el servidor emisor.

- `producer`: Identificador del componente u origen del mensaje.
- `data`: Carga útil de negocio específica con identificadores de entidades, montos y claves de trazabilidad.

P1-4.6: TOPOLOGÍA DE TÓPICOS Y GRUPOS DE CONSUMIDORES



Distribución del Trabajo de Liquidación Masiva

Para procesar más de 1.000.000 de apuestas en un plazo objetivo inferior a 15 minutos sin generar una única tarea gigantesca monolítica y sin introducir un segundo broker:

1. **Fase 1 (Cálculo Agregado Global en el Worker):**
   Al recibir el evento `ResultadoOficialPublicado`, el `WorkerLiquidacion` ejecuta una consulta agregada sobre la bolsa de la carrera en la BD operativa, determinando la masa ganadora `T_w`, la bolsa neta `B_{\text{neta}}` y la cuota unitaria base. En esta misma fase se calcula el reparto determinista aplicando la política de mayores restos a nivel global (ver P1-5.6), asegurando la conservación exacta de la bolsa antes de generar cualquier orden de acreditación.
2. **Fase 2 (Generación de Lotes y Granularidad de Mensajes):**
   - Las órdenes de acreditación se agrupan en **lotes de tamaño acotado** (por ejemplo, 500 a 1.000 apuestas premiadas por mensaje). - Cada mensaje de lote contiene un identificador único de lote (`batch_id`), el `race_id`, el rango de secuencias de apuestas incluidas y la lista de transferencias `[{ bet_id, user_id, payout_cents }]`.
3. **Fase 3 (Clave de Particionamiento y Distribución entre Particiones):**
   - Dado que un lote contiene múltiples apuestas que pueden pertenecer a diferentes usuarios, **no es posible usar `user_id` como clave a nivel de todo el lote**. - Para distribuir la carga de una carrera popular de manera uniforme entre las particiones del tópico `liquidacion.lotes`, se utiliza como clave de partición el identificador del lote: `key = batch_id` (o round-robin balanceado). Esto dispersa los lotes equitativamente entre las `P` particiones de Kafka y los workers en ejecución.
4. **Fase 4 (Procesamiento Concurrente y Orden de Bloqueo en Billeteras):**
   - Múltiples workers consumen lotes en paralelo. Para evitar interbloqueos (*deadlocks*) en la base de datos cuando dos workers procesan lotes que contienen casualmente al mismo usuario (por ejemplo, un usuario con múltiples apuestas ganadoras), el worker **ordena internamente las operaciones por `user_id` ascendente** antes de solicitar los bloqueos transaccionales (`SELECT FOR UPDATE`). - Cada acreditación individual se registra con la clave idempotente de premio basada en la apuesta (`bet_id`).
5. **Fase 5 (Reanudación de Lotes Parciales y Detección de Finalización):**
   - Si un worker colapsa a mitad de un lote, el reintento de procesamiento es seguro: las acreditaciones ya ejecutadas colisionan por clave única y se omiten (`ON CONFLICT DO NOTHING`), completándose únicamente las pendientes. - El worker emite un evento de control `LoteLiquidacionCompletado (batch_id, race_id)`. El orquestador de liquidación mantiene un contador de lotes confirmados frente al total generado; al registrarse el 100% de los lotes, la carrera transiciona formalmente a estado `LIQUIDADA`.

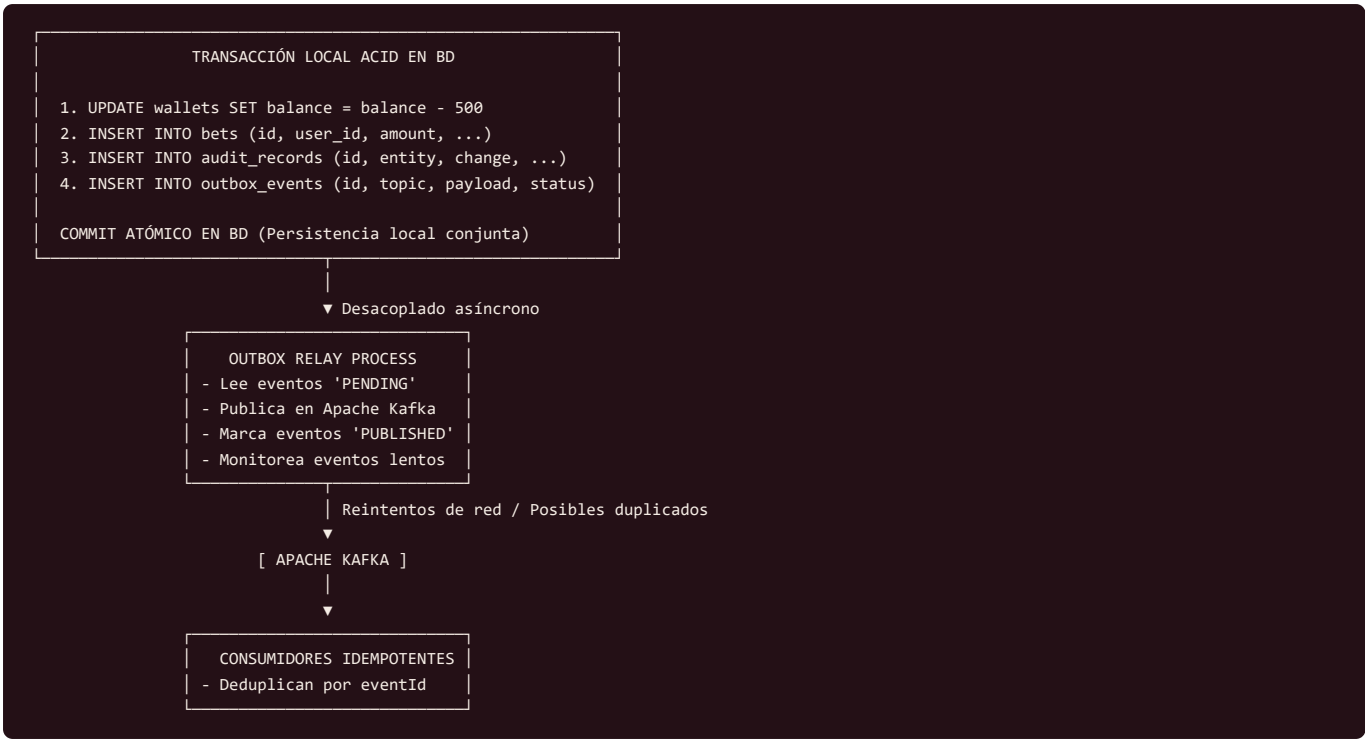
6. **Validación del Objetivo de 15 Minutos:**
   - *(Aclaración de diseño):* El particionamiento y la división en lotes estructuran el paralelismo, pero **no constituyen por sí solos una garantía matemática de terminar en menos de 15 minutos**. Cumplir esta meta bajo 1.000.000 de apuestas exige dimensionar la base de datos para soportar un rendimiento sostenido de al menos `1.111\text{ a }2.000\text{ escrituras contables/s}`, pools de conexiones dimensionados, I/O aprovisionado y validación empírica mediante pruebas de carga de estrés.

P1-4.7: ESTRATEGIA DE REINTENTOS, CONFIRMACIONES, MENSAJES DUPLICADOS Y FALLIDOS

1. **Garantía de Entrega y Tolerancia a Duplicados:**
   Kafka opera bajo la semántica de entrega **al menos una vez** (*at-least-once*). En consecuencia, **los consumidores deben ser estrictamente idempotentes**. Todo consumidor registra en su almacén local el `eventId` procesado; si un mensaje se reentrega por caída de red, el consumidor detecta el `eventId` y emite confirmación de offset sin reejecutar la lógica.
2. **Tópicos de Reintento y Carta Muerta (Dead Letter Topic - DLT):**
   - Si un consumidor falla por error transitorio (ej. micro-corte hacia el proveedor push de notificaciones), el mensaje se remite a un tópico de reintento (`notificaciones.retry-5s`) con cabecera de contador de intentos. - Si tras 3 reintentos el fallo persiste (ej. dirección de correo corrupta), se desvía al tópico de descarte (`notificaciones.dlt`) para inspección operativa sin detener el consumo del flujo principal.

P1-4.8: PUBLICACIÓN CONFIABLE: PATRÓN TRANSACTIONAL OUTBOX

Para desacoplar la persistencia en base de datos de la publicación en mensajería sin recurrir a transacciones distribuidas (2PC), se implementa el patrón **Transactional Outbox**:



Mecánica de Funcionamiento y Límites de la Garantía:

1. **Transacción Local Inicial:**
   La deducción de saldo en la billetera, el registro de la apuesta admitida, la traza local de auditoría y el registro del evento en la tabla `outbox_events` ocurren dentro de la **misma transacción atómica local en la base de datos**. No hay comunicación de red con Kafka durante esta transacción.

2. **Publicación Asíncrona Posterior por un Relay:**
   Un proceso emisor independiente (*Outbox Relay*, basado en lectura de logs de transacción / CDC o polling de alta frecuencia) lee los eventos en estado `PENDING` y los remite a Apache Kafka.
3. **Reintentos y Publicaciones Duplicadas:**
   Si el relay despacha un mensaje a Kafka pero la confirmación (*ack*) se pierde o demora, el relay reintentará el envío. Por tanto, **Kafka puede recibir eventos duplicados**. La garantía de publicación es de tipo *at-least-once*, lo que hace obligatorio que los consumidores sean estrictamente idempotentes mediante la clave `eventId`.
4. **Condiciones de Recuperación, Monitoreo y Disponibilidad:**
   - *(Aclaración crítica):* El patrón Outbox **no garantiza una entrega mágica e inmediata**. La publicación depende de la disponibilidad del broker y de la salud del proceso relay. - Si Kafka no está disponible, los eventos se acumulan de forma segura en la tabla `outbox_events` sin bloquear la admisión de apuestas; al restablecerse el broker, el relay drena los eventos pendientes en orden. - Se debe implementar monitoreo continuo sobre el retraso (*outbox lag*) y una alerta operativa si existen eventos en estado `PENDING` con una antigüedad superior a un umbral definido (por ejemplo, > 30 segundos).

ETAPA P1-5: DATOS, PERSISTENCIA E IDEMPOTENCIA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-006`
Versión: `v1.0`
Estado: `[x]` Consolidado (Políticas de negocio y correcciones técnicas incorporadas)

P1-5.1: ALMACENAMIENTO Y PROPIEDAD DE DATOS

Para cumplir con la Decisión 9 del PDF (pág. 16), se definen los tipos de bases de datos, su propiedad y sus fronteras de acceso:

1. **Base de Datos Operativa Transaccional (OLTP):** - **Tecnología recomendada:** Motor Relacional SQL (ej. PostgreSQL administrado). - **Propiedad:** Exclusiva del **Núcleo Modular Transaccional (`core-api`)**. - **Justificación:** Maneja el libro contable de billeteras, el catálogo de competidores, las carreras y las apuestas. Requiere soporte estricto de transacciones ACID, aislamiento *read committed* con bloqueos pesimistas a nivel de fila (`SELECT FOR UPDATE`), restricciones de clave foránea y claves únicas para garantizar que ningún saldo sea negativo. - **Esquema Interno:**
   - `schema_users`: Usuarios, credenciales cifradas y roles RBAC.
   - `schema_races`: Catálogo de medianos y carreras con estados y marcas temporales.
   - `schema_bets`: Apuestas admitidas con índices únicos de idempotencia.
   - `schema_wallets`: Saldos de EIA Coins y libro de movimientos contables (*ledger* append-only).
   - `schema_outbox`: Tabla transaccional de eventos pendientes de publicación en Kafka.
2. **Almacén Analítico e Histórico (OLAP):** - **Tecnología recomendada:** Almacén columnar / motor de analítica distribuida (ej. ClickHouse o servicio administrado tipo BigQuery/Snowflake). - **Propiedad:** Exclusiva del **Servicio de Analítica y Estadísticas (`analytics-service`)**. - **Justificación:** Optimizado para la ingesta masiva de millones de eventos diarios y agregaciones analíticas complejas (cálculo de tendencias, rendimiento histórico de medianos y reportes premium) sin consumir los IOPS ni el pool de conexiones de la base de datos transaccional.
3. **Almacén de Auditoría Inmutable:** - **Tecnología recomendada:** Base de datos con tablas append-only dedicadas y almacenamiento de archivos con retención de solo lectura mediante políticas de inmutabilidad (WORM / Object Lock). - **Propiedad:** Consumo exclusivo de solo lectura para el rol Auditor (`audit-service`). - **Retención:** Mínimo **5 años** conforme al requisito del PDF (pág. 9).

P1-5.2: CONSISTENCIA FUERTE VS. CONSISTENCIA EVENTUAL

Se clasifican formalmente los datos de la plataforma según su exigencia de consistencia:

Dominio de Datos	Modelo de Consistencia	Justificación Técnica	Mecanismo de Aplicación
Saldo y Movimientos de Billetera	**Consistencia Fuerte (ACID)**	Invariante de negocio: prohibición absoluta de doble gasto de EIA Coins y de saldo negativo.	Transacción local en BD con bloqueo pesimista a nivel de fila (`FOR UPDATE`).
Admisión y Cierre de Apuestas	**Consistencia Fuerte (ACID)**	Una apuesta no puede aceptarse después de la hora exacta de corte del servidor ni registrarse sin descontar el saldo.	Verificación atómica en la misma sentencia `INSERT ... WHERE NOW() < start_time`.
Cuotas Estimadas en Vivo	**Consistencia Eventual**	La cuota dinámica es una aproximación progresiva para informar al usuario mientras la carrera está abierta (PDF pág. 3).	Cálculo en streaming desacoplado mediante consumo asíncrono de eventos `ApuestaAceptada`.
Notificaciones al Apostador	**Consistencia Eventual**	El envío de correos o push puede demorarse segundos sin invalidar la confirmación transaccional ya emitida.	Consumo asíncrono desde Kafka con reintentos y tolerancia a caídas.

Dominio de Datos	Modelo de Consistencia	Justificación Técnica	Mecanismo de Aplicación
Estadísticas y Analítica	**Consistencia Eventual**	El PDF autoriza explícitamente retrasos de minutos para estadísticas generales y hasta un día para reportes premium (pág. 9).	Ingesta en micro-lotes hacia el almacén columnar OLAP.

P1-5.3: CONCURRENCIA Y PREVENCIÓN ABSOLUTA DE SALDO NEGATIVO

Bajo el escenario crítico en que un estudiante presiona cuatro veces "APOSTAR" o emite múltiples apuestas en paralelo desde varias pestañas, se previene el saldo negativo mediante tres niveles de defensa:

```
-- 1. Bloqueo exclusivo a nivel de fila sobre la billetera del usuario
SELECT balance FROM wallets WHERE user_id = :user_id FOR UPDATE;

-- 2. Validación en memoria de la transacción
-- IF balance < :amount THEN ROLLBACK AND RETURN 422_INSUFFICIENT_FUNDS;

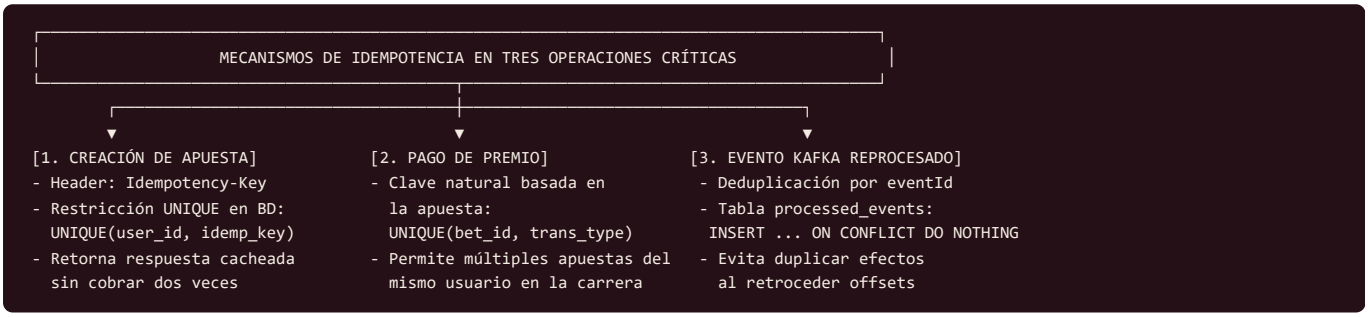
-- 3. Actualización segura y restricción CHECK a nivel de motor de base de datos
UPDATE wallets
SET balance = balance - :amount,
    updated_at = NOW()
WHERE user_id = :user_id;

-- Constraint de seguridad física en la definición de tabla:
-- ALTER TABLE wallets ADD CONSTRAINT chk_positive_balance CHECK (balance >= 0);
```

- **Explicación del Mecanismo:** La cláusula `FOR UPDATE` serializa el acceso a la fila de saldo de ese usuario específico. Las solicitudes concurrentes del mismo usuario se encolan a nivel de base de datos y se evalúan secuencialmente. Si la primera apuesta agota el saldo, las tres siguientes leen el balance actualizado y son rechazadas atómicamente con error de saldo insuficiente, disparando `ROLLBACK`. La restricción física `CHECK (balance >= 0)` garantiza a nivel de motor que jamás se confirme un valor negativo.

P1-5.4: ESTRATEGIA DE IDEMPOTENCIA EN AL MENOS TRES OPERACIONES CRÍTICAS (PDF pág. 15)

Conforme a la Decisión 6 del PDF, se diseñan los mecanismos para garantizar idempotencia (`f(f(x)) = f(x)`):



1. **Operación 1: Creación y Cobro de una Apuesta (`POST /api/v1/bets`):** - *Riesgo:* El usuario pulsa cuatro veces consecutivas el botón de apostar ante lentitud de red. - *Mecanismo:* El cliente genera una cabecera HTTP obligatoria `Idempotency-Key: <UUIDv4>`. En la base de datos existe la restricción `UNIQUE (user_id, idempotency_key)` en la tabla `bets_idempotency`. Si entra la segunda, tercera o cuarta petición con la misma clave, la inserción colisiona atómicamente; el servidor intercepta la colisión, lee la respuesta almacenada de la primera transacción confirmada y la retorna de inmediato (`HTTP 200 OK`) sin tocar el balance ni registrar una nueva apuesta.
2. **Operación 2: Acreditación de Premios por Liquidación:** - *Riesgo:* El worker de liquidación sufre un fallo transitorio tras pagar un premio y Kafka le reentrega el mensaje de lote, o un operador ejecuta un reintento manual de liquidación. - *Mecanismo y Granularidad de la Clave:* Dado que un usuario puede realizar **múltiples apuestas ganadoras legítimas en la misma carrera** (por ejemplo, dos jugadas distintas de 500 EIA Coins cada una), la clave no

puede agregarse a nivel de `(race_id, user_id)`. Se define una **clave de unicidad vinculada directamente a la apuesta premiada**:

$$\text{UNIQUE}(\text{bet_id}, \text{transaction_type})$$

donde `transaction_type = 'PRIZE_PAYOUT'`. Cada apuesta ganadora recibe su pago individual de forma exacta. Si un lote se reintenta, la base d

3. **Operación 3: Consumo de Eventos de Mensajería (Replay / At-Least-Once):** - *Riesgo:* Kafka reentrega el evento `ApuestaAceptada` tras un rebalanceo del grupo de consumidores, o un auditor retrocede el offset del tópico para inspección. - *Mecanismo:* Cada consumidor mantiene una tabla local `processed_events (event_id VARCHAR PRIMARY KEY, processed_at TIMESTAMP)`. Antes de procesar cualquier mensaje, se ejecuta: `sql INSERT INTO processed_events (event_id, processed_at) VALUES (:eventId, NOW()) ON CONFLICT (event_id) DO NOTHING;` Si la fila ya existía (afecta 0 filas), el consumidor confirma el offset en Kafka y descarta el procesamiento del mensaje duplicado de forma transparente.

P1-5.5: DISTINCIÓN ENTRE REINTENTOS E INTENCIONES LEGÍTIMAS DE NUEVAS APUESTAS

Para dar cumplimiento formal al requisito `P1-REQ-013` (un usuario puede apostar varias veces en la misma carrera, incluso por medianos distintos): - **Nueva Apuesta Legítima:** Cuando el usuario decide voluntariamente apostar 200 EIA Coins a *Frodito Turbo* y posteriormente 300 EIA Coins a *Bilbo Rápido*, la aplicación cliente genera un **nuevo y diferente** `Idempotency-Key` para cada transacción. El sistema las procesa como operaciones independientes, debitando ambas sumas y registrando dos apuestas distintas con identificadores únicos (`bet_id_1`, `bet_id_2`). - **Reintento Duplicado:** Si la red falla y la aplicación cliente reenvía la petición con la **misma** `Idempotency-Key`, el sistema la trata como un reintento idempotente, retornando la confirmación de la primera sin descontar nuevamente los fondos.

P1-5.6: POLÍTICA DETERMINISTA DE REDONDEO MEDIANTE MAYORES RESTOS (`P1-DEC-003`)

> **Aclaración institucional y contable:**
> La adopción de una **precisión contable de dos decimales** para los EIA Coins es una **decisión técnica del proyecto** para garantizar exactitud financiera, no un requisito impuesto por el PDF. Para erradicar la imprecisión de coma flotante binaria, todos los cálculos de distribución se realizan utilizando **unidades mínimas enteras (centavos de EIA Coin / centi-coins)**:

$$1\text{ EIA Coin} = 100\text{ unidades mínimas (centavos)}$$

Reglas Formales del Algoritmo de Mayores Restos:

1. **Bolsa Total Recaudada en Centavos (`B`):** Sumatoria entera exacta de los montos apostados:

$$B = \sum_{i=1}^{N} \text{monto_centavos}_i$$

2. **Determinación de la Comisión Contabilizada (`R`):** - Se calcula el 10% teórico: `R_{\text{teorico}} = 0.10 \times B`. - Como `0.10 \times B` puede no ser un entero exacto de centavos, se define la **regla determinista de redondeo bancario estándar al centavo más cercano** (`ROUND_HALF_EVEN`):

$$R = \text{round}(0.10 \times B)$$

- *(Aclaración de rigor):* Con precisión contable finita, **no es matemáticamente posible mantener exactamente el 10.000...% en todas las sumas impares de centavos**; la comisión contabilizada absorbe la regla de redondeo determinista. 3. **Bolsa Neta Distribuible Exacta (`B_{\text{neta}}`):**

- La masa total a repartir entre ganadores se define estrictamente por sustracción:

$$B_{\text{neta}} = B - R$$

- Esto garantiza por construcción que:

$$B_{\text{neta}} + R = B \quad \text{(Conservación absoluta de la bolsa)}$$

4. **Cálculo de Cuotas y Cuotas Teóricas por Apuesta Ganadora:**
   Sea T_w la masa total apostada (en centavos) al mediano ganador, y sea a_k una apuesta ganadora con monto m_k. El premio teórico continuo es:

$$P_{\text{teorico}}(a_k) = \frac{B_{\text{neta}} \times m_k}{T_w}$$

5. **Asignación de Unidades Enteras Inferiores:**
   Cada apuesta recibe inicialmente la parte entera inferior truncada de centavos:

$$P_{\text{base}}(a_k) = \lfloor P_{\text{teorico}}(a_k) \rfloor$$

El residuo fraccionario de cada apuesta es:

$$\text{resto}_k = P_{\text{teorico}}(a_k) - P_{\text{base}}(a_k)$$

6. **Cálculo de Centavos Restantes a Distribuir:**

$$C_{\text{remanente}} = B_{\text{neta}} - \sum_{k=1}^{|G|} P_{\text{base}}(a_k)$$

Donde $0 \le C_{\text{remanente}} < |G|$. 7. **Distribución Determinista de Mayores Restos:** - Se ordenan las apuestas ganadoras de mayor a menor valor de resto_k. - ***Criterio de Desempate Determinista:*** *Si dos o más apuestas presentan exactamente el mismo resto fraccionario, el desempate se resuelve ordenando de forma determinista por el* ***identificador único de apuesta en orden lexicográfico ascendente (*** bet_id***)****. Jamás se utiliza el orden de llegada de red ni la secuencia aleatoria de ejecución de workers. - A las primeras* C {\text{remanente}} apuestas de la lista ordenada se les adiciona exactamente $+1\text{ centavo}$ ($+0.01\text{ EIA Coin}$). 8. **Determinación Global Previa a la Dispersión en Lotes:**
El reparto completo se calcula en **una única fase analítica global** en el `WorkerLiquidacion` tras sellar el resultado. Una vez calculados los importes finales exactos de cada apuesta ganadora, se generan los lotes de acreditación. Los workers dispersores jamás recalculan residuos de forma aislada, garantizando que la sumatoria de pagos coincida al centavo con B_{neta}. 9. **Trazabilidad y Versión de la Política:**
La liquidación almacena en la tabla de resultados de la carrera la versión del algoritmo (`settlement_policy_version = 'HAMILTON_LARGEST_REMAINDER_V1'`), los parámetros globales (B , R , B_{neta} , T_w) y el desglose de cada apuesta para reproducción exacta por el rol Auditor.

Ejemplo Numérico Demostrativo de Conservación de la Bolsa

- **Supuesto del ejemplo:**
- Carrera con 3 apuestas ganadoras (A_1 , A_2 , A_3).
- Montos apostados: $A_1 = 33.33\text{ EIA Coins}$ (3.333 centavos), $A_2 = 33.33\text{ EIA Coins}$ (3.333 centavos), $A_3 = 33.34\text{ EIA Coins}$ (3.334 centavos).
- Masa total ganadora: $T_w = 100.00\text{ EIA Coins}$ (10.000 centavos).
- Bolsa bruta total recaudada en la carrera: $B = 100.05\text{ EIA Coins}$ (10.005 centavos).
- **Paso 1: Comisión de la casa:**
- $R_{\text{teorico}} = 0.10 \times 10.005 = 1.000,5\text{ centavos}$.
- Con redondeo bancario: $R = 1.000\text{ centavos}$ (10.00 EIA Coins).

- **Paso 2: Bolsa Neta Distribuible:**
- $B_{\text{neta}} = 10.005 - 1.000 = 9.005\text{ centavos}$ (90.05 EIA Coins).
- **Paso 3: Cálculo teórico proporcional por apuesta:**
- $P_{\text{teorico}}(A_1) = \frac{9.005 \times 3.333}{10.000} = 3.001,3665\text{ centavos} \to \text{Base} = 3.001$, $\text{Resto} = 0,3665$.
- $P_{\text{teorico}}(A_2) = \frac{9.005 \times 3.333}{10.000} = 3.001,3665\text{ centavos} \to \text{Base} = 3.001$, $\text{Resto} = 0,3665$.
- $P_{\text{teorico}}(A_3) = \frac{9.005 \times 3.334}{10.000} = 3.002,2670\text{ centavos} \to \text{Base} = 3.002$, $\text{Resto} = 0,2670$.
- **Paso 4: Suma base y residuo:**
- $\text{Suma Bases} = 3.001 + 3.001 + 3.002 = 9.004\text{ centavos}$.
- $C_{\text{remanente}} = 9.005 - 9.004 = 1\text{ centavo}$ a repartir.
- **Paso 5: Asignación por mayor resto y desempate determinista:**
- Comparación de restos: $A_1\ (0,3665)$ empata con $A_2\ (0,3665)$, y ambos superan a $A_3\ (0,2670)$.
- Desempate determinista por `bet_id`: Suponiendo que $A_1.\text{bet_id} = \text{'BET-001'}$ y $A_2.\text{bet_id} = \text{'BET-002'}$, el identificador `'BET-001'` antecede lexicográficamente.
- El centavo remanente se asigna a A_1.
- **Resultado Final:**
- $A_1 = 3.001 + 1 = 3.002\text{ centavos}$ (30.02 EIA Coins).
- $A_2 = 3.001\text{ centavos}$ (30.01 EIA Coins).
- $A_3 = 3.002\text{ centavos}$ (30.02 EIA Coins).
- **Comprobación de Conservación Absoluta:**

$$\sum \text{Premios} = 30.02 + 30.01 + 30.02 = 90.05\text{ EIA Coins}$$

$$\text{Bolsa Total} = \text{Comisión Contabilizada} (10.00) + \text{Suma Premios} (90.05) = 100.05\text{ EIA Coins} \equiv B$$

(Verificado: Cero fuga o ganancia artificial de fondos).

P1-5.7: POLÍTICA ANTE AUSENCIA DE GANADORES (`P1-DEC-004`)

> **Aclaración institucional:**
> El PDF no estipula el destino de la bolsa si ningún usuario apostó por el mediano ganador. Esta política constituye una **decisión formal del proyecto** para resolver dicho vacío normativo.

Reglas Formales ante Ausencia de Ganadores:

1. **Condición de Disparo:** Se publica el resultado oficial de la carrera y la masa de apuestas sobre el mediano ganador es nula ($T_w = 0$).
2. **Reembolso Íntegro del 100%:** Se reintegra el 100% de lo apostado a cada usuario que haya participado con apuestas aceptadas en esa carrera.
3. **No Retención de Comisión:** Salchipapa Maker Racing S.A.S. renuncia a la retención del 10% en este escenario excepcional; los apostadores reciben la totalidad de su dinero virtual.
4. **No Generación de Pozo Acumulado:** No se acumula bolsa para carreras futuras (*no rollover*), garantizando el cierre contable limpio e inmediato del evento.
5. **Preservación del Ciclo de Vida y del Historial:** - **La carrera NO pasa a estado `CANCELADA`:** La carrera sí se corrió deportivamente, el mediano ganador sí cruzó la meta y las estadísticas de rendimiento de los competidores permanecen válidas en el catálogo. - El estado de la carrera transiciona a `FINALIZADA` y posteriormente a `LIQUIDADA`. - Se registra un motivo de liquidación explícito en el registro oficial:

$$\text{settlement_reason} = \text{'SIN_APUESTAS_GANADORAS'}$$

6. **Diferenciación Conceptual de Devoluciones:**
   Se distingue formalmente la devolución por carrera suspendida/anulada (`REFUND_RACE_CANCELLED`) de la devolución por ausencia de aciertos (`REFUND_NO_WINNERS`), permitiendo al auditor distinguir incidentes operativos de desenlaces deportivos sin ganadores.
7. **Idempotencia en Devoluciones:** Cada devolución individual está indexada por la clave de unicidad `UNIQUE (bet_id, 'REFUND_NO_WINNERS')`, evitando duplicaciones contables ante reintentos de workers.
8. **Tratamiento de Bolsa Vacía:** Si una carrera no acumuló ninguna apuesta (`B = 0`), el proceso transiciona directamente a `LIQUIDADA` con motivo `BOLSA_VACIA`, sin invocar divisiones por cero ni crear registros contables artificiales.

P1-5.8: HISTÓRICO, ANALÍTICA Y AUDITORÍA INALTERABLE DE CINCO AÑOS

1. **Separación de Ciclos de Vida:**
   - Los datos operativos de apuestas en la base de datos OLTP se conservan activos durante la temporada. - Mediante un proceso de archivado periódico (ETL / streaming desde Kafka), las apuestas y transacciones se consolidan en el almacén de auditoría inmutable de largo plazo.
2. **Mecanismos de Protección Efectiva contra Alteraciones:**
   - *(Aclaración conceptual crítica):* **La replicación de bases de datos no equivale a inmutabilidad**. Si un atacante o error muta un dato en el nodo primario, la replicación propagaría la mutación a las réplicas. - La inmutabilidad real de los 5 años se fundamenta en tres barreras:
   1. *Permisos de Motor:* Conexiones de auditoría configuradas con privilegios exclusivos `SELECT` a nivel de RBAC de base de datos.
   2. *Almacenamiento Físico WORM / Object Lock:* Los registros históricos consolidados se exportan en formato inmutable (ej. Parquet/ORC) a un almacenamiento de objetos con política WORM (*Write Once, Read Many*) con bloqueo de retención de cumplimiento legal por 5 años, que impide físicamente la eliminación o modificación incluso por administradores de sistema.
   3. *Sellado Criptográfico:* Cadenas de hashes criptográficos (SHA-256) generadas al cierre de cada carrera, permitiendo verificar matemáticamente que las tablas de auditoría no han sufrido adulteración física.
3. **Reconstrucción Forense de Premios:**
   - Para cualquier carrera histórica, el rol Auditor puede consultar el registro congelado: monto de la bolsa `B`, mediano ganador oficial certificado, masa `T_w`, cuota final `Q_w`, versión de política utilizada y lista exhaustiva de apuestas premiadas, reproduciendo exactamente el algoritmo de mayores restos para verificar la exactitud de cada EIA Coin distribuido.

P1-5.9: JUSTIFICACIÓN DE TRANSACCIONES LOCALES, OUTBOX Y SAGAS

- **Transacciones Locales (ACID):** Se utilizan en la ruta crítica del Núcleo Modular para asegurar la atomicidad entre el débito de saldo, la validación de cierre y el registro de la apuesta.
- **Transactional Outbox:** Se utiliza para la publicación confiable de eventos hacia Apache Kafka, garantizando que no existan eventos "fantasma" si la base de datos aborta la transacción, ni eventos perdidos si el broker sufre una interrupción transitoria.
- **Descarte de Sagas en la Ruta Crítica:** Se justifica explícitamente no utilizar el patrón Saga para la aceptación de apuestas, dado que requeriría llamadas remotas asíncronas entre apuestas y billetera con transacciones de compensación que harían extremadamente complejo asegurar el límite de 1 segundo e impedir saldos negativos concurrentes.
- **Saga Coreografiada para Liquidación de Premios:** La liquidación masiva adopta un flujo desacoplado por etapas fuera de la ruta crítica: `ResultadoPublicado` `\to` `CalcularPremiosGlobal` `\to` `DispersarLotes` `\to` `ConfirmarLiquidacion`, con reintentos idempotentes y trazabilidad individualizada por apuesta.

ETAPA P1-6: ESCALABILIDAD, DIMENSIONAMIENTO Y MANEJO DE FALLOS

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-007`
Versión: `v1.0`
Estado: `[x]` Consolidado y verificado por revisión autónoma

P1-6.1 Y P1-6.2: ANÁLISIS DE CARGA MASIVA, CUELLOS DE BOTELLA Y ESCALAMIENTO

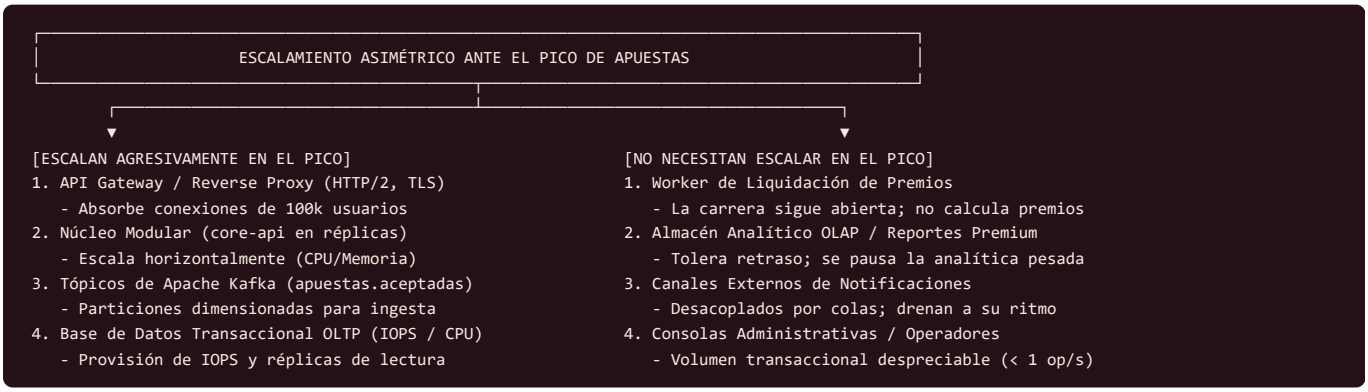
1. Perfil Cuantitativo de la Carga Extrema (PDF págs. 8, 15)

- **Carga Concurrente Máxima:** 100.000 usuarios conectados en la final de la Copa Intersemestral Dyllan Salchipapa Maker.
- **Ráfaga Transaccional de Apuestas: 10.000 apuestas por segundo sostenidas durante los 30 segundos previos al cierre**.
- **Volumen Total del Pico:** $10.000\text{ apuestas/s} \times 30\text{ s} = 300.000\text{ apuestas}$ procesadas bajo máxima contención.

2. Identificación Precisa de Cuellos de Botella Físicos

Bajo esta ráfaga masiva, los puntos críticos de saturación no se ubican en la capa de cómputo stateless, sino en: 1. **Contención de Conexiones en Borde:** 100.000 conexiones concurrentes demandan terminación TLS eficiente y balanceo con *connection pooling* para no agotar los descriptores de archivo del sistema operativo. 2. **Contención de Base de Datos en Bloqueos de Fila:** 10.000 escrituras concurrentes por segundo sobre la tabla de billeteras. Aunque cada usuario tiene su propia fila (`wallets`), si múltiples apuestas concurrentes provienen del mismo usuario (o de bots), se genera contención por la cláusula `SELECT FOR UPDATE`. 3. **Escritura del Write-Ahead Log (WAL) e IOPS de Almacenamiento:** Confirmar 10.000 transacciones ACID locales por segundo en disco físico impone una demanda masiva de IOPS y rendimiento de sincronización de fsync.

3. Asimetría de Escalamiento: Componentes que Escalan vs. Componentes que No Escalan Igual



- **Componentes que deben escalar agresivamente:**
- *API Gateway:* Escalamiento horizontal elástico para manejar los 100.000 sockets y aplicar rate limiting/anti-bot.
- *Instancias del Núcleo Modular (`core-api`):* Pool de réplicas sin estado (*stateless*) para procesar validaciones de tokens, reglas de negocio e inserciones locales.
- *Almacenamiento OLTP:* Aprovisionamiento elástico de IOPS en almacenamiento SSD/NVMe y réplicas de lectura para segregar consultas.
- **Componentes que NO escalan en el pico:**
- *Liquidación de Premios:* Permanece inactiva durante la ráfaga (la carrera no ha finalizado).
- *Servicio de Analítica:* Mantiene consumo a ritmo diferido sin demandar recursos durante el cierre.
- *Módulos Administrativos:* El tráfico de operadores y auditores es mínimo y no requiere escalamiento elástico.

P1-6.3: CUMPLIMIENTO DEL TIEMPO DE RESPUESTA EN MENOS DE UN SEGUNDO

Para asegurar que el usuario conozca deterministamente si su jugada fue aceptada o rechazada en `< 1\text{ segundo}` bajo ráfaga: 1. **Presupuesto de Latencia (*Latency Budget*):** - Red de cliente a Gateway (Red externa 4G/WiFi): `\approx 150 - 300\text{ ms}`. - Procesamiento en Gateway (Rate limiting, auth token check): `\approx 5 - 10\text{ ms}`. - Red privada interna (Gateway a `core-api`): `\approx 2 - 5\text{ ms}`. - Transacción Local Atómica en BD (Bloqueo de fila por usuario + validación de hora + inserts): `\approx 15 - 40\text{ ms}`. - Serialización y respuesta HTTP 201: `\approx 5\text{ ms}`. - Retorno por red al cliente: `\approx 150 - 300\text{ ms}`. - **Latencia Total Estimada:** `\mathbf{330 - 660\text{ ms}}` (holgadamente inferior al umbral de 1.000 ms). 2. **Estrategias para Proteger el Presupuesto de Latencia:** - Cero saltos de red distribuidos síncronos en el backend (se rechazaron microservicios con llamadas RPC intermedias). - Publicación asíncrona mediante Transactional Outbox: la escritura en Kafka no forma parte del ciclo de reloj de la petición HTTP del cliente.

P1-6.4: MECANISMO TÉCNICO PARA IMPEDIR ACEPTACIONES POSTERIORES AL CIERRE

Conforme a la regla del PDF (pág. 3: "*Una apuesta que llegue después del cierre debe rechazarse, aunque el usuario la haya enviado antes*"): 1. **Autoridad Absoluta del Reloj:** Se implementa sincronización temporal mediante Network Time Protocol (NTP / PTP) en todos los nodos de cómputo y base de datos con deriva inferior a 5 ms. 2. **Filtro de Dos Fases:** - *Fase 1 (En memoria en `core-api`):* Si `local_server_time >= race.start_time`, la petición se rechaza de inmediato con `HTTP 409 Conflict` sin interactuar con la base de datos, aliviando la carga del motor en los segundos posteriores al corte. - *Fase 2 (Atómica en BD - Guardián Definitivo):* La sentencia de inserción condiciona su ejecución a la marca temporal local del motor: `sql INSERT INTO bets (id, user_id, race_id, racer_id, amount, created_at) SELECT :betId, :userId, :raceId, :racerId, :amount, CLOCK_TIMESTAMP() FROM races WHERE races.id = :raceId AND races.status = 'ABIERTA' AND CLOCK_TIMESTAMP() < races.start_time;` Si el reloj del motor alcanzó o superó `races.start_time`, la consulta afecta 0 filas, provocando el aborto transaccional determinista.

P1-6.5: CUMPLIMIENTO DEL PLAZO DE LIQUIDACIÓN (< 15 MINUTOS PARA > 1.000.000 DE APUESTAS)

Para liquidar más de 1.000.000 de apuestas en menos de 15 minutos (900 segundos): 1. **Demanda Contable Mínima:**

```
\text{Throughput Mínimo} = \frac{1.000.000\text{ apuestas}}{900\text{ segundos}} \approx 1.111\text{
                                    acreditaciones por segundo}
```

2. **Dimensionamiento y Paralelismo:** - La fase global del algoritmo de mayores restos toma `< 10\text{ segundos}` en memoria en el `WorkerLiquidacion`. - Se generan `\sim 1.000\text{ a } 2.000\text{ lotes}` de 500 a 1.000 apuestas cada uno en el tópico `liquidacion.lotes`. - Un pool de **10 workers concurrentes** consumiendo particiones independientes ejecuta inserciones masivas agrupadas (`batch inserts` y `UPDATE wallets ... FROM (VALUES ...)`), logrando una capacidad agregada de `2.000\text{ a } 3.000\text{ acreditaciones/s}`. - **Tiempo Total Estimado:** `\mathbf{6\text{ a } 9\text{ minutos}}` (satisfaciendo holgadamente la exigencia de 15 minutos).

P1-6.6: DURABILIDAD ABSOLUTA Y RECUPERACIÓN ANTE DESASTRES

Para dar cumplimiento formal al requisito `P1-REQ-008` (pérdida cero de apuestas aceptadas y movimientos de billetera): 1. **Persistencia Síncrona en Motor OLTP:** Configuración de base de datos con `synchronous_commit = on` y replicación síncrona a nivel de *Write-Ahead Log* hacia al menos una réplica secundaria en zona de disponibilidad independiente. 2. **Puntos de Recuperación (*RPO* y *RTO*):** - **RPO (Recovery Point Objective): 0 segundos** (cero pérdida de transacciones confirmadas). - **RTO (Recovery Time Objective): < 30 segundos** mediante conmutación por error automática (*failover*) del

nodo primario de base de datos. 3. **Respaldo Físico y Continuo:** Copias de seguridad completas diarias más archivado continuo de segmentos WAL (*Point-in-Time Recovery* - PITR) hacia almacenamiento de objetos redundante.

P1-6.7 Y P1-6.8: ANÁLISIS FORMAL DE ESCENARIOS DE FALLO (PDF pág. 16)

Caso A: El sistema de notificaciones está caído

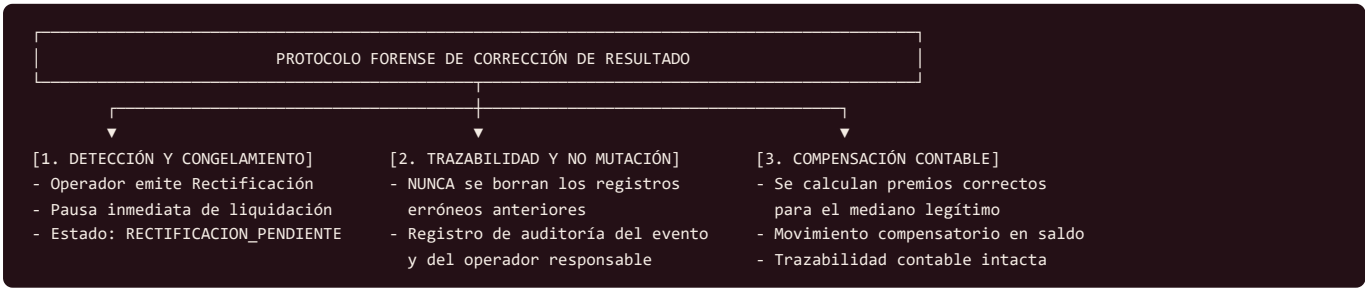
- **¿Los usuarios deberían poder seguir apostando?: SÍ, TOTALMENTE.**
- **Justificación Técnica:**
  La aceptación de una apuesta es un hecho transaccional crítico que ocurre exclusivamente entre el cliente, el API Gateway, el Núcleo Modular y la base de datos OLTP. El subsistema de notificaciones está **completamente desacoplado de la ruta crítica** mediante Apache Kafka.
- **Comportamiento del Sistema:**
  Los eventos `ApuestaAceptada` continúan acumulándose de forma segura en el tópico de Kafka. El usuario recibe su confirmación síncrona en pantalla (`HTTP 201 Created`). Cuando el servicio de notificaciones se recupera, reanuda la lectura desde su último offset confirmado y despacha los correos/push pendientes sin haber afectado en absoluto la disponibilidad operativa de las apuestas.

Caso B: El servicio de billetera (o módulo de billetera) está caído

- **¿Los usuarios deberían poder seguir apostando?: NO, DEBEN RECHAZARSE / BLOQUEARSE.**
- **Justificación Técnica:**
  La comprobación de saldo y el descuento de EIA Coins constituyen una **precondición obligatoria e indivisible** para admitir una apuesta (invariante de no saldo negativo y no doble gasto).
- **Explicación de por qué las respuestas son diametralmente distintas:**
  Mientras que las notificaciones son una capacidad informativa de **consistencia eventual** (cuya demora o pérdida temporal no altera el estado financiero de los participantes), la billetera gobierna la **consistencia fuerte y la integridad de fondos** de la plataforma. Admitir apuestas con la billetera inhabilitada obligaría a operar a crédito o a ciegas, derivando en saldos negativos incontrolados y violando las reglas fundamentales del modelo pari-mutuel.

P1-6.9: CASO C (OPCIONAL): CORRECCIÓN DE RESULTADOS TRAS FOTO DE LLEGADA MAL INTERPRETADA

Conforme al caso opcional del PDF (pág. 16): *Diez minutos después de publicado el resultado oficial, un operador descubre que la foto de llegada fue mal interpretada y el ganador era otro mediano, habiéndose pagado ya algunos premios*.



Protocolo de Compensación sin Pérdida de Trazabilidad:

1. **Prohibición de Alteración Destructiva:**
   Bajo ninguna circunstancia se ejecutan sentencias destructivas (`DELETE` o `UPDATE` sobre el historial contable ya aplicado). La auditoría forense exige preservar la evidencia del error humano original.
2. **Transición a Estado `RESULTADO_EN_RECTIFICACION`:**
   El operador autorizado registra formalmente la corrección (`POST /api/v1/races/{id}/rectify`) adjuntando la nueva evidencia visual y el motivo. El sistema frena de inmediato cualquier lote de acreditación en curso para esa carrera.

3. **Cálculo de la Nueva Liquidación Oficial:**
   El worker recalcula la bolsa mutua para el mediano legítimo M_{correcto}, generando los montos de premio correctos con una nueva versión de liquidación (`settlement_version = 2`).
4. **Política de Saldo y Ajuste Contable:** - **Invariante de Saldo Preservado:** Si el usuario que recibió el premio erróneo ya consumió sus EIA Coins, **su balance jamás se fuerza a negativo**, respetando la restricción física `CHECK (balance >= 0)`. No se inventa una recuperación mágica de fondos virtuales inexistentes. - **Diferenciación de Movimientos:**
   - *Premio Original:* Registrado con `transaction_type = 'PRIZE_PAYOUT'` y `settlement_version = 1`.
   - *Nuevo Premio Legítimo:* Se abona a los verdaderos ganadores con `transaction_type = 'PRIZE_PAYOUT'` y `settlement_version = 2`. La clave de idempotencia compuesta `UNIQUE(bet_id, transaction_type, settlement_version)` permite el pago legítimo sin colisionar con la liquidación errónea previa.
   - *Ajuste Compensatorio:* Al usuario beneficiado por error se le descuenta el balance disponible hasta un mínimo de 0 (`LEAST(balance, erroneous_amount)`) mediante un movimiento de tipo `ADJUSTMENT_ERRONEOUS_PAYOUT`. El importe insoluto no recuperable se registra en auditoría como pérdida operativa de Salchipapa Maker Racing asumida por rectificación deportiva.
5. **Sellado de Auditoría:**
   Se emite un evento inmutable `ResultadoRectificado` que ingresa a la cadena criptográfica de auditoría, documentando la identidad del operador, la marca de tiempo de la corrección y el impacto financiero exacto.

P1-6.10: PLAN CONCEPTUAL DE MÉTRICAS Y VALIDACIÓN ARQUITECTÓNICA

Para validar empíricamente que la arquitectura soporta las demandas cuantitativas antes de la Copa Intersemestral:

Dimensión a Validar	Métrica Clave	Umbral Aceptable	Herramienta / Método Conceptual
Capacidad de Ingesta	Tasa de apuestas aceptadas/s	$\ge 10.000\text{ apuestas/s}$ sostenidas durante 30 s	Prueba de carga distribuida en ambiente de pruebas (Locust / k6).
Latencia Percibida	Percentil 95 y 99 (`p95, p99`)	$p99 < 800\text{ ms}$ en respuesta de confirmación	Trazabilidad distribuida y métricas de APM (Prometheus + Grafana).
Integridad de Saldo	Conteo de saldos negativos	**Exactamente 0** bajo ráfaga concurrente	Verificación contable automatizada tras prueba de estrés.
Cierre Estricto	Apuestas aceptadas con $t \ge T_{\text{cierre}}$	**Exactamente 0**	Inyección de tráfico cruzando deliberadamente la marca de tiempo de corte.
Plazo de Liquidación	Duración de liquidación masiva	$< 15\text{ minutos}$ para $1.000.000$ de apuestas simuladas	Medición de reloj entre evento `ResultadoOficialPublicado` y `CarreraLiquidada`.

ETAPA P1-7: SEGURIDAD, CONTROL DE ACCESO Y GOBERNANZA DE COSTOS

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-008`
Versión: `v1.0`
Estado: `[x]` Consolidado y verificado por revisión autónoma

P1-7.1: AUTENTICACIÓN, AUTORIZACIÓN Y MATRIZ RBAC (PDF pág. 4, 12)

Para proteger las operaciones que involucran EIA Coins y credenciales de usuario, se diseña un esquema de seguridad perimetral basado en estándares de la industria:

1. **Autenticación Basada en Tokens Criptográficos (JWT Stateless):** - El usuario se autentica mediante `POST /api/v1/auth/login`. Tras validar el hash seguro de la contraseña (Argon2id / bcrypt), el servicio emite un token de acceso firmado digitalmente (EdDSA / RS256) con tiempo de expiración corto (15 a 30 minutos) y un *refresh token* opaco de rotación persistido de forma segura. - El token contiene las credenciales de identidad (`sub: user_id`) y el rol asignado (`role`), permitiendo a las réplicas del Núcleo Modular validar la firma y permisos localmente en memoria sin consultar a la base de datos en cada una de las 10.000 req/s.
2. **Matriz de Control de Acceso Basado en Roles (RBAC):**

Capacidad / Endpoint	Apostador	Operador de Carrera	Administrador	Auditor
Consultar carreras y catálogo de medianos	Permitido	Permitido	Permitido	Permitido
Emitir apuesta (`POST /bets`)	**Permitido**	Denegado	Denegado	Denegado
Consultar saldo y movimientos propios	**Permitido**	Denegado	Denegado	Denegado
Programar carrera y pistas (`POST /races`)	Denegado	**Permitido**	Permitido	Denegado
Publicar resultado oficial (`POST /results`)	Denegado	**Permitido**	Denegado	Denegado
Bloquear / Desbloquear usuarios	Denegado	Denegado	**Permitido**	Denegado
Consultar registros de auditoría y trazas	Denegado	Denegado	Solo métricas	**Solo Lectura Exclusivo**
Modificar saldos o apuestas directamente	**Prohibido**	**Prohibido**	**Prohibido**	**Prohibido**

P1-7.2: PROTECCIÓN DE APIS, LIMITACIÓN DE TASA (RATE LIMITING) Y ANTI-BOT

En la final de la Copa Intersemestral, el pico de 10.000 apuestas/s puede ser agravado por scripts maliciosos o usuarios recargando compulsivamente. Se establecen mecanismos de protección perimetral en el API Gateway: 1. **Rate Limiting Multinivel (Algoritmo Token Bucket / Leaky Bucket):** - *Por Usuario Autenticado:* Límite de **5 apuestas por segundo por usuario** (con ráfagas de hasta 10). Suficiente para permitir apuestas múltiples legítimas en distintos medianos, pero bloqueando scripts agresivos que monopolicen el pool de conexiones. - *Por Dirección IP:* Límite sobre endpoints públicos de consulta (`GET /races`, `GET /odds`) de 50 peticiones por minuto por IP para mitigar raspado (*scraping*) de cuotas. - *Respuesta a Excesos:* HTTP `429 Too Many Requests` con cabecera `Retry-After: <segundos>`. 2. **Mitigación de Bots y Apuestas Masivas al Cierre:** - Detección en tiempo real de huellas digitales de dispositivos y firmas de cliente sospechosas. - Bloqueo preventivo de múltiples cuentas autenticadas operando concurrentemente desde una misma IP en los últimos 10 segundos antes del corte.

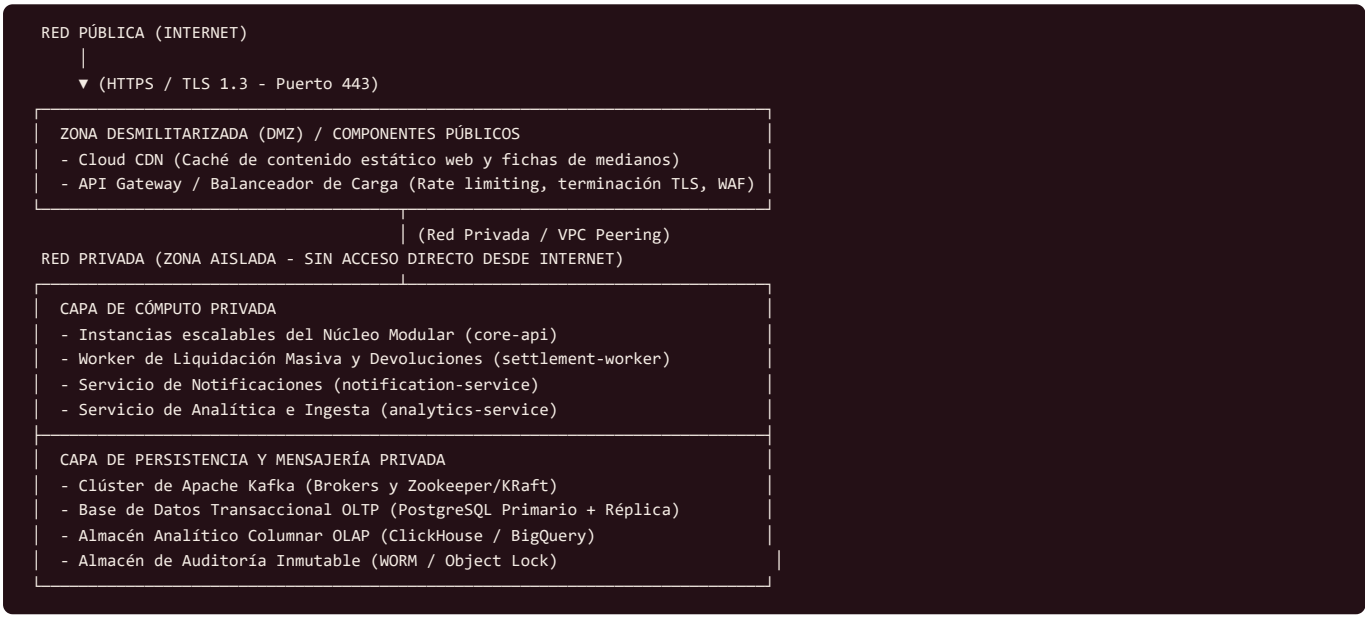
P1-7.3: GESTIÓN DE CIFRADO Y CREDENCIALES

1. **Cifrado en Tránsito:** - Todo el tráfico externo (cliente hacia API Gateway) exige **TLS 1.3** estricto con renovación automática de certificados. - El tráfico interno en la red privada (entre Gateway, `core-api`, Kafka y base de datos) utiliza mTLS (*Mutual TLS*) o red virtual aislada (VPC/VNet con cifrado nativo de infraestructura).
2. **Cifrado en Reposo:** - Bases de datos operativas y almacenes analíticos operan con cifrado de volumen a nivel de disco (AES-256). - Datos personales sensibles y claves de billetera se almacenan en reposo cifrados mediante claves custodiadas en un servicio de gestión de secretos (*KMS / Secret Manager*).
3. **Manejo Seguro de Credenciales:**
   Prohibición absoluta de almacenar secretos, cadenas de conexión o tokens en el código fuente, archivos del repositorio o variables de entorno expuestas. Las credenciales se inyectan en tiempo de ejecución desde un gestor de secretos administrado.

P1-7.4: REGISTRO DE ACCIONES ADMINISTRATIVAS Y AUDITORÍA FORENSE (PDF pág. 12)

Toda mutación administrativa u operativa que afecte saldos, usuarios o estados de carrera genera una traza obligatoria: - **Campos del Registro de Auditoría:** `audit_id`, `operator_id`, `action_type`, `resource_id`, `before_state`, `after_state`, `ip_address`, `timestamp`. - **Inalterabilidad de la Traza:** La inserción de la auditoría administrativa ocurre en la misma transacción local o se despacha hacia el log seguro append-only. Ningún usuario, incluido el Administrador, cuenta con privilegios SQL de `UPDATE` o `DELETE` sobre las tablas de auditoría.

P1-7.5: SEGMENTACIÓN DE RED: COMPONENTES PÚBLICOS VS. PRIVADOS (PDF pág. 12)



- **Componentes Públicos:** Exclusivamente el CDN y el API Gateway/WAF. Ningún microservicio ni base de datos expone puertos directos a Internet.
- **Componentes Privados:** Todas las instancias de cómputo (`core-api`, workers, analítica), el clúster de Kafka y las bases de datos transaccionales y analíticas residen en subredes privadas sin IP pública accesible.

P1-7.6 Y P1-7.7: GOBERNANZA DE COSTOS, SERVICIOS ADMINISTRADOS Y JUSTIFICACIÓN (PDF pág. 12-13)

1. Cumplimiento de la Directriz de Dyllan ("No pagar infraestructura de Netflix")

El diseño arquitectónico adoptado erradica activamente la sobreingeniería: - **Consolidación en 1 Solo Servicio Core Replicable:** En lugar de operar 9 microservicios con 9 bases de datos, registros de servicios y balanceadores independientes, el Núcleo Modular agrupa las capacidades transaccionales en un único artefacto desplegable (`core-api`). - **Broker Único:** Un solo clúster de Apache Kafka cubre streaming, analítica, auditoría y lotes de liquidación, evitando la duplicidad de costes de operar RabbitMQ simultáneamente. - **Servicios Administrados Justificados:**
Es razonable utilizar servicios administrados (ej. Base de Datos Administrada y Kafka Administrado) para transferir la carga de alta disponibilidad, parches de seguridad, conmutación por error y respaldos automáticos, reduciendo a cero la necesidad de un equipo de operaciones dedicado 24/7. - **Apagado y Elasticidad en Horas Valle:** La carga extrema solo ocurre durante torneos semestrales específicos. La infraestructura de cómputo escala a un mínimo económico (ej. 2 instancias pequeñas) en operación normal de 500 usuarios, y escala elásticamente solo durante los días de competencia de la Liga.

ADR-001: Selección del Estilo Arquitectónico

Proyecto: La Gran Liga de Apuestas de Medianos EIA
Empresa: Salchipapa Maker Racing S.A.S.
Estado: `[x]` Aprobado por el usuario
Fecha: 2026-10-27

---

1. Contexto y Problema

Salchipapa Maker Racing S.A.S. requiere diseñar la arquitectura de la plataforma de apuestas mutuas sobre carreras de medianos. El sistema debe gestionar una carga base de 500 usuarios y absorber picos de hasta 100.000 usuarios concurrentes con ráfagas de 10.000 apuestas por segundo durante los 30 segundos previos al cierre. La solución debe informar la aceptación o rechazo de cada apuesta en menos de 1 segundo, garantizar que ningún saldo quede negativo ni se duplique un débito, asegurar el cierre estricto según el reloj del servidor y cumplir la regla directiva de Dyllan: "*No quiero pagar una infraestructura de Netflix para una carrera de cinco medianos*".

2. Opciones Consideradas

1. **Monolito Tradicional:** Sistema acoplado en un único proceso desplegable con base de datos compartida y sin fronteras modulares estrictas.
2. **Monolito Modular Completo:** Todas las capacidades agrupadas en un único artefacto desplegable replicable horizontalmente, estructurado internamente en módulos con límites y contratos de interfaz formales.
3. **Microservicios Puros:** Descomposición completa de las capacidades en 9 servicios autónomos con bases de datos privadas (*database-per-service*) y comunicación distribuida síncrona/asíncrona.
4. **Arquitectura Híbrida (Núcleo Modular Transaccional + Servicios Asíncronos Desacoplados):** Núcleo modular replicable para el flujo transaccional cohesivo (usuarios, medianos, carreras, apuestas, billetera), complementado con servicios/workers autónomos para procesos asíncronos desacoplados (notificaciones, analítica masiva y liquidación de premios).

3. Decisión Tomada

Se adopta la **Arquitectura Híbrida**. El flujo crítico de admisión y cobro se concentra en un Núcleo Modular Transaccional desplegado en instancias replicables, permitiendo diseñar la validación de saldo, el débito y el registro de la apuesta dentro de una misma transacción local atómica (ACID). Las capacidades de Notificaciones, Analítica/Estadísticas y Liquidación masiva se segregan en unidades y procesos especializados fuera de la ruta crítica de aceptación.

4. Justificación del Descarte de Alternativas

- **Descarte de Microservicios Puros:** La separación física de apuestas y billetera impondría coordinación distribuida (patrón Saga con transacciones compensatorias) en la ruta crítica, aumentando la exposición a inconsistencias y saldos negativos bajo concurrencia masiva. Asimismo, la sobrecarga operativa y de infraestructura asociada contraviene la política de costo y simplicidad exigida por la dirección.
- **Descarte del Monolito Tradicional:** La ausencia de límites físicos permitiría que la saturación en proveedores externos de notificación o la carga pesada de reportes analíticos degrade el hilo transaccional y bloquee la admisión de apuestas, violando el requisito de aislamiento de fallos.

5. Consecuencias

Lo que se gana:

- **Consistencia atómica simplificada:** Eliminación de protocolos de coordinación distribuida (2PC o Sagas) en el flujo crítico de débito y apuesta.

- **Aislamiento efectivo de fallas:** La caída o lentitud de notificaciones no impacta la recepción ni confirmación de apuestas.
- **Aislamiento de carga analítica:** Millones de eventos analíticos se ingieren y consultan en un almacén optimizado sin canibalizar recursos transaccionales.
- **Gobernanza de costos:** Infraestructura moderada, mantenible y proporcional a la escala de la organización.

Lo que se sacrifica y límites:

- **Disciplina modular obligatoria:** Requiere control riguroso del código para impedir dependencias circulares o violaciones de encapsulamiento entre módulos del núcleo.
- **Contención en almacenamiento compartido:** Al replicar el núcleo transaccional en múltiples instancias, el cuello de botella se traslada a la base de datos operativa, exigiendo particionamiento por usuario, bloqueos a nivel de fila y separación estricta de lecturas y escrituras que deberán validarse en las etapas de persistencia y escalabilidad.

ADR-002: Selección de la Tecnología de Mensajería y Event Streaming

Proyecto: La Gran Liga de Apuestas de Medianos EIA
Empresa: Salchipapa Maker Racing S.A.S.
Estado: `[x]` Aprobado por el usuario
Fecha: 2026-10-27

---

1. Contexto y Problema

La plataforma de apuestas sobre carreras de medianos requiere una infraestructura asíncrona para propagar eventos transaccionales desde el núcleo hacia múltiples subsistemas independientes: cálculo en vivo de cuotas dinámicas, notificación multicanal a apostadores, alimentación del almacén de analítica y auditoría/control de fraude. Asimismo, se debe coordinar la liquidación masiva de premios de más de 1.000.000 de apuestas en menos de 15 minutos sin bloquear el tráfico interactivo y respetando la directriz de costo y parsimonia técnica de la organización.

2. Opciones Consideradas

1. **RabbitMQ como Broker Único:** Sistema basado en colas AMQP, exchanges y enrutamiento flexible, complementable con colas tradicionales o streams.
2. **Apache Kafka como Broker Único:** Plataforma distribuida de streaming de eventos basada en log de confirmación particionado (*commit log* append-only) con persistencia configurable y grupos de consumidores independientes.
3. **Solución Dual (RabbitMQ + Kafka):** Kafka para la ingestión masiva de analítica y eventos de apuestas; RabbitMQ para la orquestación de tareas de liquidación y notificaciones.
4. **Ninguno (Comunicación Síncrona Directa):** Acoplamiento directo mediante llamadas REST/gRPC entre todos los componentes.

3. Decisión Tomada

Se adopta **Apache Kafka como tecnología única de mensajería y event streaming**. Los eventos transaccionales del núcleo (apuestas, inicios, finales y resultados) se publican en tópicos particionados mediante el patrón Transactional Outbox. Múltiples grupos de consumidores independientes (`Consumer Groups`) leen concurrentemente las particiones a su propio ritmo sin duplicación física de datos en el broker. La liquidación masiva de premios se resuelve mediante tareas en lotes particionados por usuario (`user_id`) sobre tópicos de trabajo en el mismo clúster.

4. Justificación del Descarte de Alternativas

- **Descarte de RabbitMQ como broker único:** Aunque cuenta con extensiones modernas como streams, el modelo nativo de Kafka de log distribuido particionado por clave de negocio (`race_id`, `user_id`) con consumidores desacoplados por offset resuelve de manera más homogénea y directa las necesidades de difusión masiva hacia múltiples subsistemas y la distribución concurrente de lotes de liquidación.
- **Descarte de la Solución Dual:** Para el diseño inicial del sistema no se ha demostrado una ventaja técnica suficiente que justifique la sobrecarga operativa, cognitiva y financiera de aprovisionar, asegurar y monitorear dos plataformas de mensajería dispares simultáneamente.
- **Descarte de Ninguno:** Descartado por el riesgo inminente de colapso en cascada bajo el pico de 10.000 apuestas/s.

5. Consecuencias

Lo que se gana:

- **Consumo desacoplado y concurrente:** Notificaciones lentas, analítica pesada y auditoría consumen del mismo log sin saturar la memoria del broker ni interferir entre sí.
- **Distribución eficiente de liquidación:** Descomposición del millón de apuestas en lotes homogéneos particionados por usuario, procesados en paralelo por workers para cumplir el plazo de 15 minutos sin generar interbloqueos de

saldo.
- **Persistencia y reprocesamiento operativo:** Retención temporal de eventos (ej. 7 días) que permite recuperar caídas de consumidores y reprocesar información reciente.

Lo que se sacrifica y límites:

- **Complejidad de operación de Kafka:** Demanda gestión de particiones, replicación y balanceo de consumidores, recomendándose el uso de servicios administrados.
- **Semántica at-least-once:** Kafka puede reentregar eventos duplicados ante reintentos de red; **los consumidores deben ser estrictamente idempotentes** para impedir dobles cobros o acreditaciones repetidas.
- **Delimitación de retención:** Kafka no sustituye el almacenamiento de auditoría inalterable a 5 años; los eventos deben persistirse en un repositorio inmutable dedicado.

ADR-003: Consistencia Transaccional, Idempotencia y Modelo de Liquidación

Proyecto: La Gran Liga de Apuestas de Medianos EIA
Empresa: Salchipapa Maker Racing S.A.S.
Estado: `[x]` Aprobado por el usuario
Fecha: 2026-10-27

---

1. Contexto y Problema

La plataforma maneja EIA Coins bajo una ráfaga masiva de hasta 10.000 apuestas/s en los 30 segundos previos al cierre, exigiendo respuesta en menos de 1 segundo sin incurrir jamás en saldo negativo ni doble gasto. Se requiere además procesar la liquidación masiva de más de 1.000.000 de apuestas en menos de 15 minutos, garantizar idempotencia ante múltiples pulsaciones del usuario o reintentos de red, distribuir los residuos fraccionarios de la bolsa común de forma justa y matemáticamente exacta, y definir el comportamiento del sistema cuando ningún apostador acierta al mediano ganador.

2. Opciones Consideradas

1. **Transacciones Distribuidas (2PC / XA):** Bloqueo en dos fases coordinando apuestas y billetera en bases de datos separadas.
2. **Patrón Saga en Ruta Crítica:** Aceptación eventual con llamadas asíncronas entre apuestas y billetera mediante transacciones compensatorias.
3. **Consistencia Local ACID en Núcleo Modular + Outbox + Algoritmo de Mayores Restos:** Validación de cierre, descuento de saldo y persistencia de apuesta en una misma transacción local en base de datos OLTP. Publicación asíncrona mediante Transactional Outbox. Reparto de premios mediante el algoritmo de mayores restos en centavos enteros con desempate determinista, y reembolso del 100% ante ausencia de ganadores sin alterar el ciclo deportivo de la carrera.

3. Decisión Tomada

Se adopta la **Opción 3**. La ruta crítica de admisión de apuestas opera bajo consistencia fuerte ACID local con bloqueos pesimistas a nivel de fila (`SELECT FOR UPDATE`) y restricción `CHECK (balance >= 0)`. La idempotencia se garantiza a tres niveles: en apuestas mediante `Idempotency-Key` en cabecera HTTP e índice `UNIQUE(user_id, idempotency_key)`, en premios mediante clave individual por apuesta `UNIQUE(bet_id, 'PRIZE_PAYOUT')`, y en consumidores Kafka mediante tabla de eventos procesados. Para el reparto de premios se adopta una precisión contable de dos decimales (centavos enteros) y el algoritmo de mayores restos con desempate determinista por `bet_id`, asegurando que la suma de premios y comisión coincida al centavo con la bolsa recaudada. Ante ausencia de ganadores, se devuelve el 100% de los fondos sin retener comisión, manteniendo el historial deportivo de la carrera con motivo `SIN_APUESTAS_GANADORAS`.

4. Consecuencias

Lo que se gana:

- **Integridad contable absoluta:** Imposibilidad matemática de saldo negativo o duplicación de pagos por reintentos de workers o cliente.
- **Conservación exacta de la bolsa:** Cero pérdida o creación espuria de centavos; la regla determinista de mayores restos es 100% reproducible por el auditor.
- **Rendimiento predecible en borde:** Respuesta transaccional local en decenas de milisegundos sin latencias de red distribuidas.
- **Transparencia en devoluciones:** Diferenciación clara entre carreras canceladas y eventos deportivos válidos sin aciertos de usuarios.

Lo que se sacrifica y límites:

- **Contención en cuentas activas:** Usuarios que apuestan masivamente en paralelo serializan sus transacciones por el bloqueo de fila de su billetera.
- **Cálculo previo centralizado:** La liquidación exige una fase analítica global para determinar los mayores restos antes de generar los lotes de dispersión.
- **Complejidad de almacenamiento inmutable:** La retención de auditoría a 5 años requiere políticas WORM y sellado criptográfico en almacenes dedicados fuera del motor OLTP.

DIAGRAMA GENERAL DE ARQUITECTURA

DIAGRAMA GENERAL DE ARQUITECTURA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Entregable Oficial: Punto 3 del PDF (págs. 17, 18)
Artefacto: `P1-ART-009A`

```
graph TD
    %% Estilos de Nodos
    classDef clientStyle fill:#F4ECE4,stroke:#100D10,stroke-width:2px,color:#100D10;
    classDef edgeStyle fill:#6B0F1A,stroke:#100D10,stroke-width:2px,color:#F4ECE4;
    classDef coreStyle fill:#8B1E2D,stroke:#100D10,stroke-width:2px,color:#F4ECE4;
    classDef dbStyle fill:#540B13,stroke:#100D10,stroke-width:2px,color:#F4ECE4;
    classDef brokerStyle fill:#100D10,stroke:#C6A66B,stroke-width:2px,color:#F4ECE4;
    classDef asyncWorker fill:#3D080E,stroke:#C6A66B,stroke-width:2px,color:#F4ECE4;

    %% 1. CLIENTES (Dispositivos Externos)
    subgraph CLIENTES ["👥 CAPA DE CLIENTES (PÚBLICA)"]
        C1["Apostador (Web / App Móvil)"]:::clientStyle
        C2["Operador de Carrera (Consola Web)"]:::clientStyle
        C3["Auditor (Consola de Solo Lectura)"]:::clientStyle
    end

    %% 2. BORDE Y SEGURIDAD (DMZ)
    subgraph BORDE ["🛡️ ZONA DESMILITARIZADA (DMZ)"]
        GW["API Gateway / WAF / Reverse Proxy<br/>- Terminación TLS 1.3<br/>- Rate Limiting (Token Bucket)<br/>- Validación Criptográfica JWT"]
    end

    %% 3. NÚCLEO MODULAR TRANSACCIONAL (VPC Privada)
    subgraph CORE ["🏛️ NÚCLEO TRANSACCIONAL MODULAR (core-api en Réplicas Stateless)"]
        direction TB
        M_AUTH["Módulo Usuarios & Auth<br/>(RBAC / Claims)"]:::coreStyle
        M_RACERS["Módulo Medianos<br/>(Catálogo / Fichas)"]:::coreStyle
        M_RACES["Módulo Carreras<br/>(Ciclo / Reloj Servidor)"]:::coreStyle
        M_BETS["Módulo Apuestas<br/>(Admisión / Idempotencia)"]:::coreStyle
        M_WALLET["Módulo Billetera<br/>(EIA Coins / Débito)"]:::coreStyle
        M_OUTBOX["Módulo Outbox<br/>(Tabla de Eventos)"]:::coreStyle
    end

    %% 4. PERSISTENCIA TRANSACCIONAL
    subgraph PERSISTENCIA ["💾 CAPA DE PERSISTENCIA OPERATIVA (OLTP)"]
        DB_PRIMARY[("PostgreSQL Primario (ACID)<br/>- Bloqueo FOR UPDATE<br/>- Inserción Atómica Apuesta<br/>- Tabla outbox_events")]:::dbStyle
        DB_REPLICA[("PostgreSQL Réplica Lectura<br/>- Cuotas Estimadas<br/>- Catálogo de Medianos")]:::dbStyle
    end

    %% 5. MENSAJERÍA Y STREAMING
    subgraph MENSAJERIA ["⚡ APACHE KAFKA (BROKER DISTRIBUIDO)"]
        T_BETS["Topic: apuestas.aceptadas<br/>(Key: race_id)"]:::brokerStyle
        T_RACES["Topic: carreras.eventos<br/>(Key: race_id)"]:::brokerStyle
        T_SETTLE["Topic: liquidacion.lotes<br/>(Key: batch_id)"]:::brokerStyle
    end

    %% 6. COMPONENTES Y CONSUMIDORES DESACOPLADOS
    subgraph ASINCRONOS ["⚙️ COMPONENTES Y WORKERS FUERA DE LA RUTA CRÍTICA"]
        W_SETTLE["Worker de Liquidación Masiva<br/>- Algoritmo Mayores Restos<br/>- Generador de Lotes"]:::asyncWorker
        W_DISPERS["Workers Dispersores de Balance<br/>- Acreditación en Lotes<br/>- Idempotencia UNIQUE(bet_id)"]:::asyncWorker
        S_NOTIF["Servicio de Notificaciones<br/>- Correo, Push, App<br/>- Falla NO bloquea Core"]:::asyncWorker
        S_ANALYTICS["Servicio de Analítica & Stats<br/>- Ingesta Diaria Millones Regs<br/>- Almacén Columnar OLAP"]:::asyncWorker
        S_AUDIT["Almacén Inmutable de Auditoría<br/>- Retención 5 Años WORM<br/>- Hashes SHA-256"]:::asyncWorker
    end

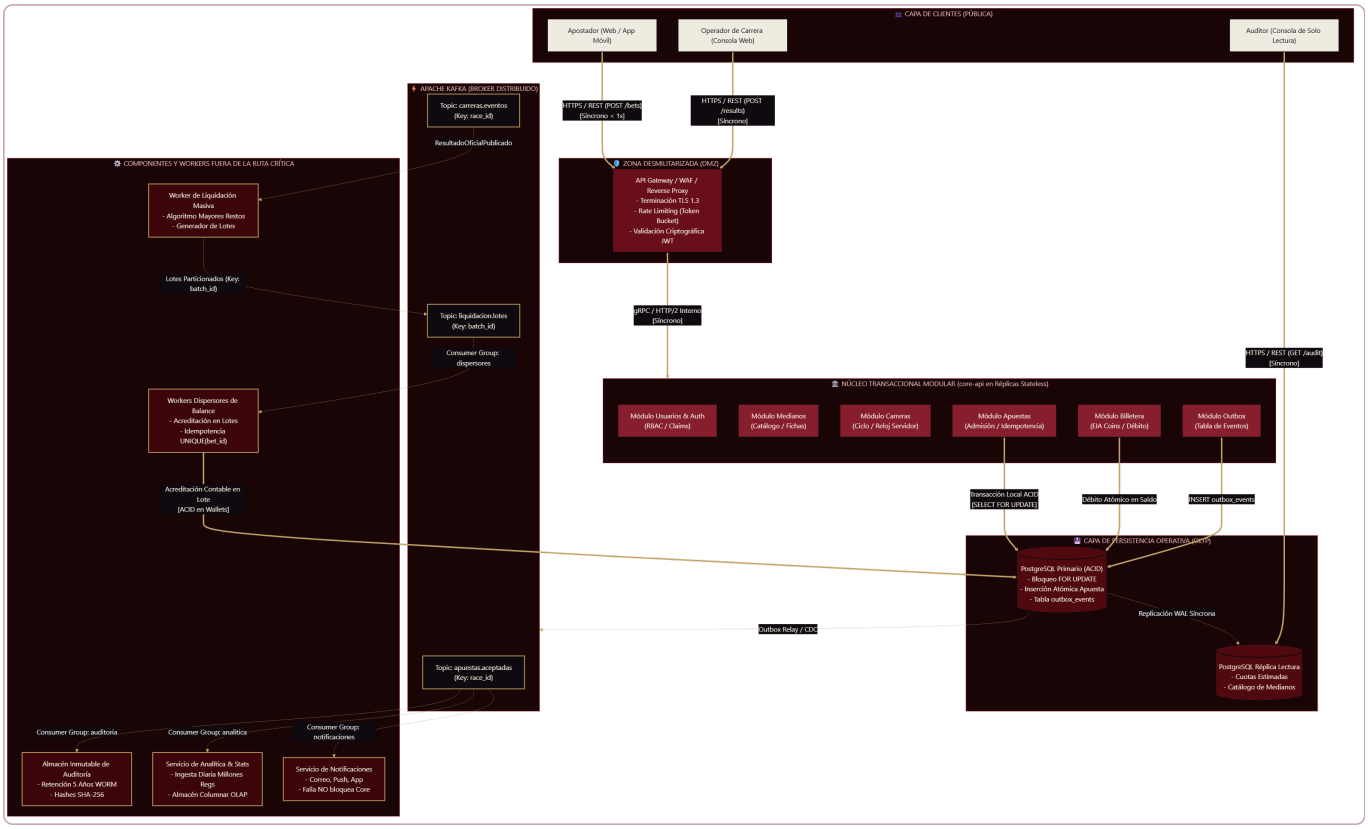
    %% Relaciones Síncronas (Flecha Sólida ===)
    C1 == "HTTPS / REST (POST /bets)<br/>[Síncrono < 1s]" ===> GW
    C2 == "HTTPS / REST (POST /results)<br/>[Síncrono]" ===> GW
    C3 == "HTTPS / REST (GET /audit)<br/>[Síncrono]" ===> DB_REPLICA
    GW == "gRPC / HTTP/2 Interno<br/>[Síncrono]" ===> CORE
    M_BETS == "Transacción Local ACID<br/>[SELECT FOR UPDATE]" ===> DB_PRIMARY
    M_WALLET == "Débito Atómico en Saldo" ===> DB_PRIMARY
    M_OUTBOX == "INSERT outbox_events" ===> DB_PRIMARY
    DB_PRIMARY -.->|"Replicación WAL Síncrona"| DB_REPLICA

    %% Relaciones Asíncronas (Flecha Punteada -.-)
    DB_PRIMARY -.->|"Outbox Relay / CDC"| MENSAJERIA
    T_BETS -.->|"Consumer Group: notificaciones"| S_NOTIF
    T_BETS -.->|"Consumer Group: analitica"| S_ANALYTICS
    T_BETS -.->|"Consumer Group: auditoria"| S_AUDIT
    T_RACES -.->|"ResultadoOficialPublicado"| W_SETTLE
    W_SETTLE -.->|"Lotes Particionados (Key: batch_id)"| T_SETTLE
    T_SETTLE -.->|"Consumer Group: dispersores"| W_DISPERS
    W_DISPERS == "Acreditación Contable en Lote<br/>[ACID en Wallets]" ===> DB_PRIMARY
```

Leyenda y Diferenciación Visual Exigida (PDF pág. 17):

- **Línea Sólida (`===`): Comunicación Síncrona** en la ruta crítica del usuario y operaciones administrativas interactivas.
- **Línea Punteada (`-.->`): Comunicación Asíncrona** basada en eventos a través de Apache Kafka y procesos en segundo plano.

Render del diagrama



Render generado desde la fuente canónica `08_diagramas_y_adrs/diagrama_general.md` (Mermaid; regenerable con `node herramientas/generar_diagramas.mjs`).

DIAGRAMA DE SECUENCIA: FLUJO COMPLETO DE UNA APUESTA

DIAGRAMA DE SECUENCIA: FLUJO COMPLETO DE UNA APUESTA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Entregable Oficial: Punto 4 del PDF (págs. 17, 18)
Artefacto: `P1-ART-009B`

```
sequenceDiagram
    autonumber
    actor U as 👤 Apostador (Cliente)
    participant GW as 🛡️ API Gateway / WAF
    participant CORE as 🏛️ Núcleo Modular (core-api)
    participant DB as 💾 Base de Datos OLTP (PostgreSQL)
    participant RELAY as ⚙️ Outbox Relay
    participant KAFKA as ⚡ Apache Kafka
    participant NOTIF as 📬 Servicio Notificaciones

    %% 1. Inicio de la Petición
    U->>+GW: POST /api/v1/bets<br/>{raceId, racerId, amount}<br/>Header: Idempotency-Key

    %% 2. Filtros Perimetrales
    Note over GW: 1. Rate Limiting (Token Bucket)<br/>2. Validación JWT & Claims RBAC

    %% 3. Enrutamiento al Núcleo
    GW->>+CORE: Enrutamiento gRPC/HTTP-2 con contexto

    %% 4. Transacción Local ACID
    Note over CORE,DB: INICIO DE TRANSACCIÓN LOCAL ACID
    CORE->>DB: BEGIN TRANSACTION

    %% 5. Verificación de Idempotencia
    CORE->>DB: SELECT response FROM bets_idempotency WHERE key = :idemp_key
    alt Clave Ya Procesada (Reintento de Red)
        DB-->>CORE: Retorna respuesta previa
        CORE-->>GW: HTTP 200 OK (Cacheada)
        GW-->>U: HTTP 200 OK (Apuesta ya confirmada)
    end

    %% 6. Validación de Cierre Atómico
    CORE->>DB: SELECT status, start_time FROM races WHERE id = :raceId
    alt Hora Servidor >= Hora Inicio Carrera O Estado != 'ABIERTA'
        DB-->>CORE: Carrera Cerrada
        CORE->>DB: ROLLBACK
        CORE-->>GW: HTTP 409 Conflict (CARRERA_CERRADA)
        GW-->>U: HTTP 409 Rechazada: Fuera de Tiempo
    end

    %% 7. Bloqueo de Billetera y Verificación de Saldo
    CORE->>DB: SELECT balance FROM wallets WHERE user_id = :uid FOR UPDATE
    alt Saldo Insuficiente (balance < amount)
        DB-->>CORE: Saldo Insuficiente
        CORE->>DB: ROLLBACK
        CORE-->>GW: HTTP 422 Unprocessable Entity
        GW-->>U: HTTP 422 Rechazada: Saldo Insuficiente
    end

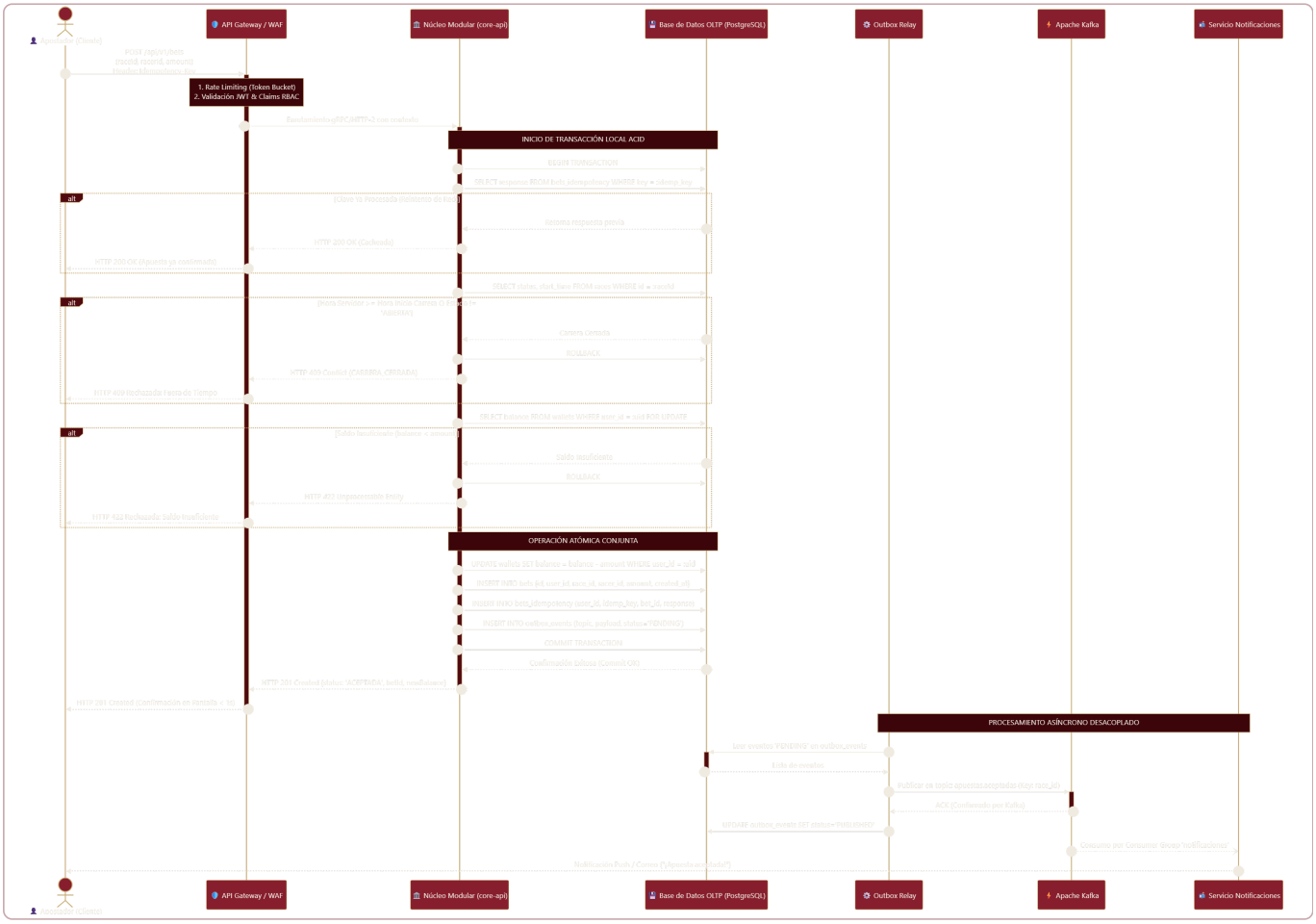
    %% 8. Débito, Registro y Outbox en la Misma Transacción
    Note over CORE,DB: OPERACIÓN ATÓMICA CONJUNTA
    CORE->>DB: UPDATE wallets SET balance = balance - amount WHERE user_id = :uid
    CORE->>DB: INSERT INTO bets (id, user_id, race_id, racer_id, amount, created_at)
    CORE->>DB: INSERT INTO bets_idempotency (user_id, idemp_key, bet_id, response)
    CORE->>DB: INSERT INTO outbox_events (topic, payload, status='PENDING')

    %% 9. Commit Local y Respuesta Inmediata
    CORE->>DB: COMMIT TRANSACTION
    DB-->>CORE: Confirmación Exitosa (Commit OK)
    CORE-->>-GW: HTTP 201 Created {status: 'ACEPTADA', betId, newBalance}
    GW-->>-U: HTTP 201 Created (Confirmación en Pantalla < 1s)

    %% 10. Flujo Asíncrono Desacoplado (Fuera de la Ruta Crítica)
    Note over RELAY,NOTIF: PROCESAMIENTO ASÍNCRONO DESACOPLADO
    RELAY->>+DB: Leer eventos 'PENDING' en outbox_events
    DB-->>-RELAY: Lista de eventos
    RELAY->>+KAFKA: Publicar en topic: apuestas.aceptadas (Key: race_id)
    KAFKA-->>-RELAY: ACK (Confirmado por Kafka)
    RELAY->>DB: UPDATE outbox_events SET status='PUBLISHED'

    KAFKA--)NOTIF: Consumo por Consumer Group 'notificaciones'
    NOTIF-->>U: Notificación Push / Correo ("¡Apuesta aceptada!")
```

Render del diagrama



Render generado desde la fuente canónica `08_diagramas_y_adrs/diagrama_secuencia_apuesta.md` (Mermaid; regenerable con `node herramientas/generar_diagramas.mjs`).

TOPOLOGÍA DE MENSAJERÍA: APACHE KAFKA

TOPOLOGÍA DE MENSAJERÍA: APACHE KAFKA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Entregable Oficial: Punto 7 del PDF (págs. 14, 18)
Artefacto: `P1-ART-009C`

```
graph TD
    %% Estilos
    classDef prodStyle fill:#8B1E2D,stroke:#100D10,stroke-width:2px,color:#F4ECE4;
    classDef topicStyle fill:#100D10,stroke:#C6A66B,stroke-width:2px,color:#F4ECE4;
    classDef groupStyle fill:#540B13,stroke:#C6A66B,stroke-width:2px,color:#F4ECE4;
    classDef consumerStyle fill:#3D080E,stroke:#F4ECE4,stroke-width:1px,color:#F4ECE4;

    %% Productores
    subgraph PRODUCTORES ["🚀 PRODUCTORES (PRODUCERS)"]
        P_CORE["Núcleo Modular / Outbox Relay<br/>(Transactional Outbox)"]:::prodStyle
        P_SETTLE["Worker de Liquidación<br/>(Generador de Lotes)"]:::prodStyle
    end

    %% Tópicos y Particiones
    subgraph TOPICS ["⚡ CLÚSTER DE APACHE KAFKA (TOPICS & PARTITIONS)"]
        subgraph T1 ["Topic: apuestas.aceptadas (8 Particiones | Key: race_id)"]
            T1_P["P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7"]:::topicStyle
        end
        subgraph T2 ["Topic: carreras.eventos (4 Particiones | Key: race_id)"]
            T2_P["P0 | P1 | P2 | P3"]:::topicStyle
        end
        subgraph T3 ["Topic: liquidacion.lotes (8 Particiones | Key: batch_id)"]
            T3_P["P0 | P1 | P2 | P3 | P4 | P5 | P6 | P7"]:::topicStyle
        end
        subgraph T4 ["Topic: notificaciones.dlt (Dead Letter Topic)"]
            T4_P["DLT - Mensajes Fallidos tras 3 Reintentos"]:::topicStyle
        end
    end

    %% Grupos de Consumidores
    subgraph GRUPOS ["👥 GRUPOS DE CONSUMIDORES (CONSUMER GROUPS)"]
        subgraph CG_CUOTAS ["Group: cuotas-dinamicas-workers"]
            C_CUOTAS["Workers de Cálculo de Cuotas en Vivo<br/>(Lectura secuencial por carrera)"]:::consumerStyle
        end
        subgraph CG_NOTIF ["Group: notificaciones-service"]
            C_NOTIF["Consumidores Multicanal<br/>(Despacho Push / Correo / App)"]:::consumerStyle
        end
        subgraph CG_ANALYTICS ["Group: analitica-ingesta-diaria"]
            C_ANALYTICS["Consumidores Micro-Lotes<br/>(Alimentación Almacén OLAP)"]:::consumerStyle
        end
        subgraph CG_AUDIT ["Group: auditoria-inmutable"]
            C_AUDIT["Consumidores de Auditoría<br/>(Sellado WORM / Hashes SHA-256)"]:::consumerStyle
        end
        subgraph CG_DISPERS ["Group: dispersores-premios-billetera"]
            C_DISPERS["Workers Acreditadores de Balance<br/>(Abono atómico con orden por user_id)"]:::consumerStyle
        end
    end

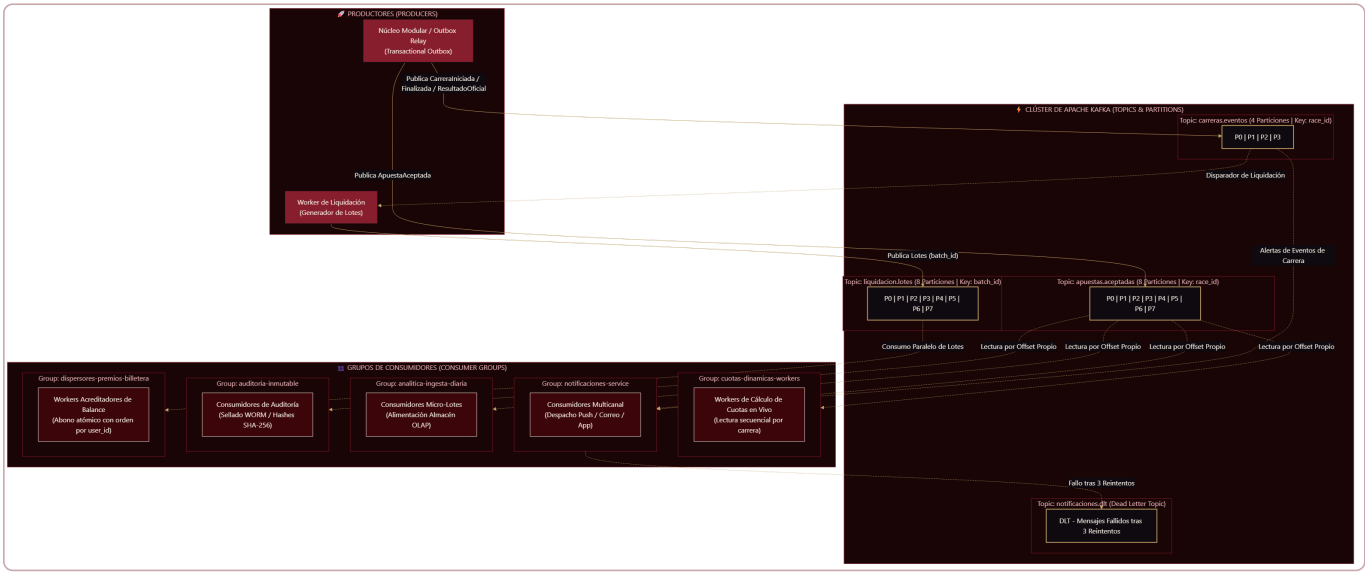
    %% Flujos de Publicación
    P_CORE -->|"Publica ApuestaAceptada"| T1_P
    P_CORE -->|"Publica CarreraIniciada / Finalizada / ResultadoOficial"| T2_P
    P_SETTLE -->|"Publica Lotes (batch_id)"| T3_P

    %% Flujos de Consumo Independiente (Offsets Desacoplados)
    T1_P -.->|"Lectura por Offset Propio"| CG_CUOTAS
    T1_P -.->|"Lectura por Offset Propio"| CG_NOTIF
    T1_P -.->|"Lectura por Offset Propio"| CG_ANALYTICS
    T1_P -.->|"Lectura por Offset Propio"| CG_AUDIT

    T2_P -.->|"Disparador de Liquidación"| P_SETTLE
    T2_P -.->|"Alertas de Eventos de Carrera"| CG_NOTIF

    T3_P -.->|"Consumo Paralelo de Lotes"| CG_DISPERS
    CG_NOTIF -.->|"Fallo tras 3 Reintentos"| T4_P
```

Render del diagrama



Render generado desde la fuente canónica `08_diagramas_y_adrs/topologia_mensajeria.md` (Mermaid; regenerable con `node herramientas/generar_diagramas.mjs`).

DIAGRAMA DE CICLO DE VIDA DE UNA CARRERA

DIAGRAMA DE CICLO DE VIDA DE UNA CARRERA

Caso: La Gran Liga de Apuestas de Medianos EIA
Organización: Salchipapa Maker Racing S.A.S.
Artefacto: `P1-ART-009D`
Uso: Apoyo visual de la presentación (Slide 04) y del documento técnico. Fuente complementaria de `01_requisitos_y_negocio.md` (P1-1.2).

```
stateDiagram-v2
    direction LR
    [*] --> PROGRAMADA : Creación y programación

    PROGRAMADA --> ABIERTA : Apertura de ventana de apuestas
    ABIERTA --> CERRADA : Hora de inicio (reloj del servidor)
    CERRADA --> EN_CURSO : Bandera de salida
    EN_CURSO --> FINALIZADA : Cruzar la meta
    FINALIZADA --> LIQUIDADA : Resultado oficial + liquidación de premios
    LIQUIDADA --> [*]

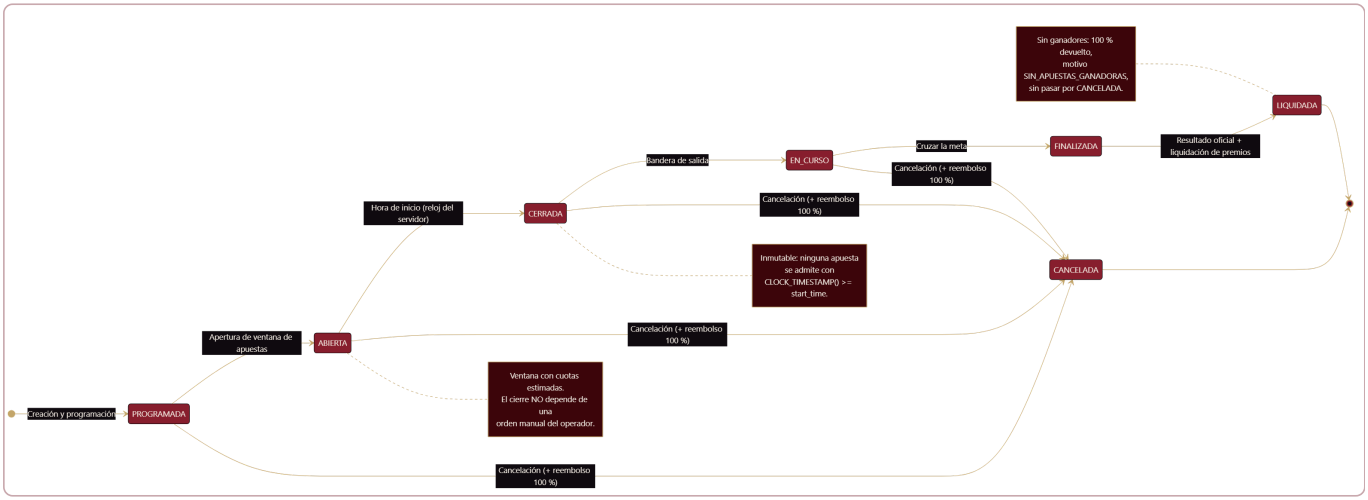
    PROGRAMADA --> CANCELADA : Cancelación (+ reembolso 100 %)
    ABIERTA --> CANCELADA : Cancelación (+ reembolso 100 %)
    CERRADA --> CANCELADA : Cancelación (+ reembolso 100 %)
    EN_CURSO --> CANCELADA : Cancelación (+ reembolso 100 %)
    CANCELADA --> [*]

    note right of ABIERTA
        Ventana con cuotas estimadas.
        El cierre NO depende de una
        orden manual del operador.
    end note
    note right of CERRADA
        Inmutable: ninguna apuesta
        se admite con
        CLOCK_TIMESTAMP() >= start_time.
    end note
    note left of LIQUIDADA
        Sin ganadores: 100 % devuelto,
        motivo SIN_APUESTAS_GANADORAS,
        sin pasar por CANCELADA.
    end note
```

Reglas representadas (Fuente: `01_requisitos_y_negocio.md`, PDF pág. 3):

- Ciclo formal: `PROGRAMADA → ABIERTA → CERRADA → EN_CURSO → FINALIZADA → LIQUIDADA`.
- `CANCELADA` es alcanzable desde cualquier estado anterior a `FINALIZADA`, con devolución íntegra del 100 %.
- La cancelación es un incidente operativo; la ausencia de ganadores es un desenlace deportivo válido (la carrera llega a `LIQUIDADA`).

Render del diagrama



Render generado desde la fuente canónica `08_diagramas_y_adrs/ciclo_carrera.md` (Mermaid; regenerable con `node herramientas/generar_diagramas.mjs`).